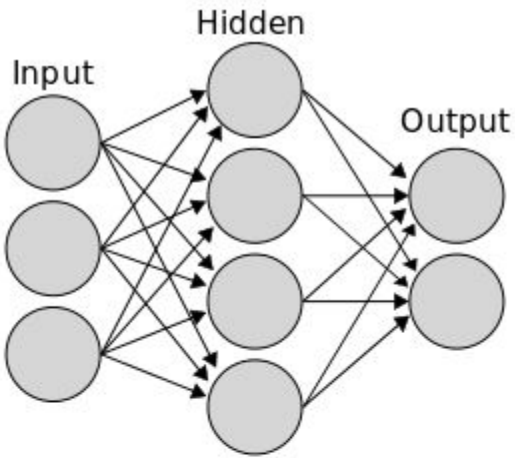
# Convolutional Neural Networks

Module 2

# Convolution Operation

# Motivation for Convolution Operation (CNN)

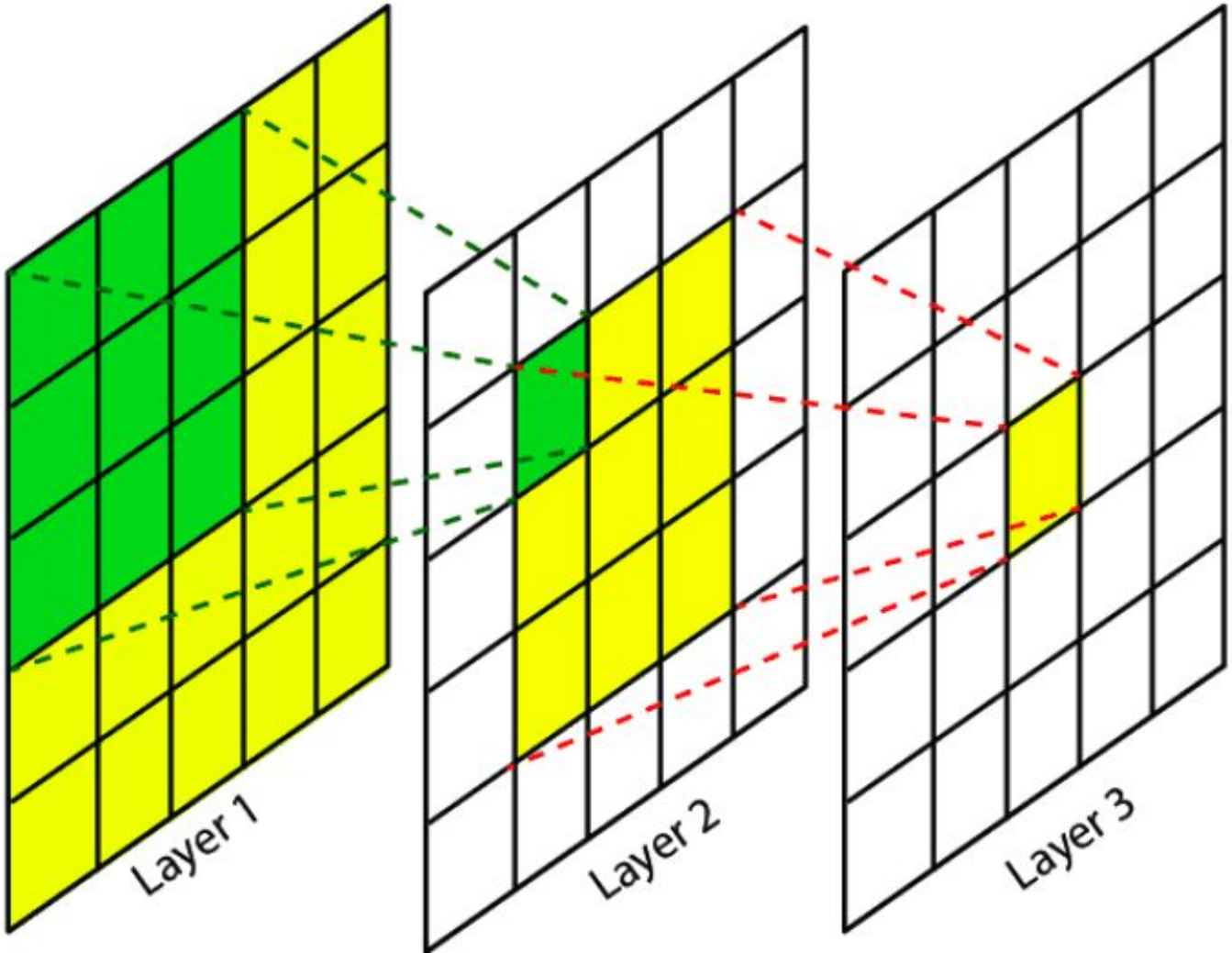
- Assume a small image of size 32×32×3
- The number of inputs are 32x32x3=3072
- If the first hidden layer has just 100 neurons:
  - 3072×100=307,200 parameters
- Problems
  - Too many parameters
  - High memory and computation
  - Severe overfitting with limited data



- Images Have Spatial Structure
  - Pixels are not independent.
  - Nearby pixels are highly correlated
  - Edges, corners, textures are local patterns
  - Example:
    - An edge is formed by a small group of pixels, not the whole image
  - Fully connected layers ignore spatial locality

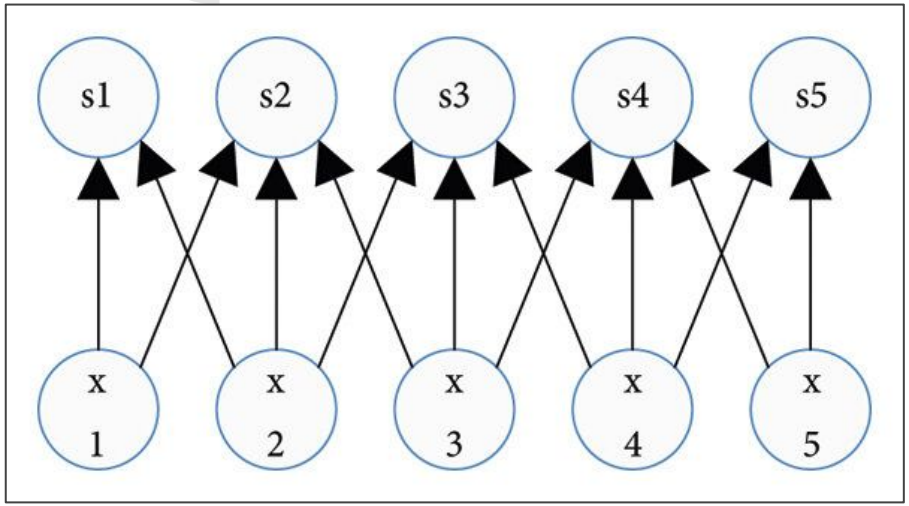
# Local Connectivity

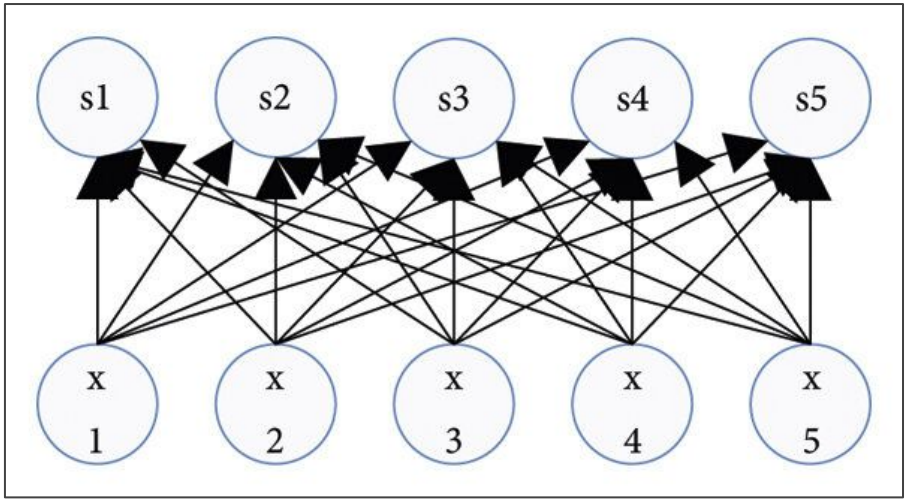
- Instead of connecting every neuron to the entire image:
- Connect each neuron to a small local region
- This region is called a receptive field.
- Example:
  - 3×3 or 5×5 patch
- Captures local features
- Fewer parameters


Layer 1
Layer 2
Layer 3

- An edge detector that works at the top-left corner:
- Should also work at the bottom-right
- So instead of learning separate weights for each location:
  - Use the same filter everywhere
  - This is called parameter sharing

# Parameter sharing Vs Fully Connected



# Translation Equivariance

- If an object shifts in the image:
- The feature map shifts in the same way
  - $\text{Shift}(x) \Rightarrow \text{Shift}(f(x))$
- This is essential for:
  - Object detection
  - Recognition
- Fully connected networks do not have this property.

# Massive Reduction in Parameters (core motivation)

- Example:
  - Filter size: 5×5
  - Input channels: 3
  - Parameters per filter: 5×5×3=75
- Compare:
  - FC layer: hundreds of thousands
  - Convolution: tens
- Less overfitting
- Faster training
- Scales to large images

# Biological Inspiration

- In the visual cortex:
  - Neurons respond to local regions
  - Same type of neurons detect similar patterns across the visual field
- CNNs mimic this behavior.

# Summary

- Fully connected neural networks are inefficient for image data due to the large number of parameters and their inability to exploit spatial structure.
- Images contain local and repeating patterns such as edges and textures.
- Convolution addresses this by using local receptive fields and parameter sharing, significantly reducing parameters while preserving spatial relationships and enabling translation equivariance.

Convolution is motivated by the need to efficiently learn local, spatially invariant features from high-dimensional structured data.

# What is Convolution?

- Mathematical Definition (2D Discrete Convolution)
  - For input $X$ and kernel $K$:

$$Y(i,j) = \sum_m \sum_n X(i+m, j+n)\, K(m,n)$$

- $X$: input image / feature map
- $K$: kernel / filter
- $Y$: output feature map

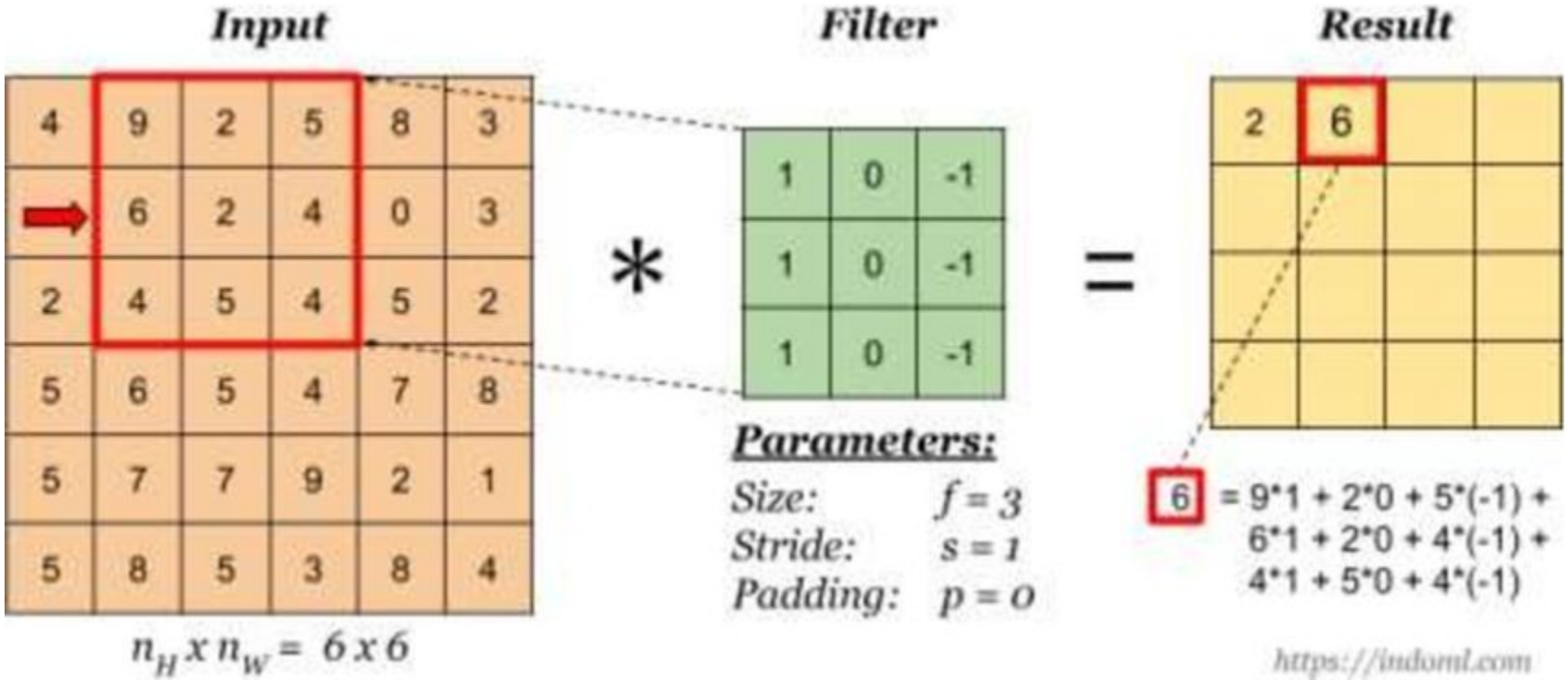
(CNNs actually use cross-correlation, not true convolution)

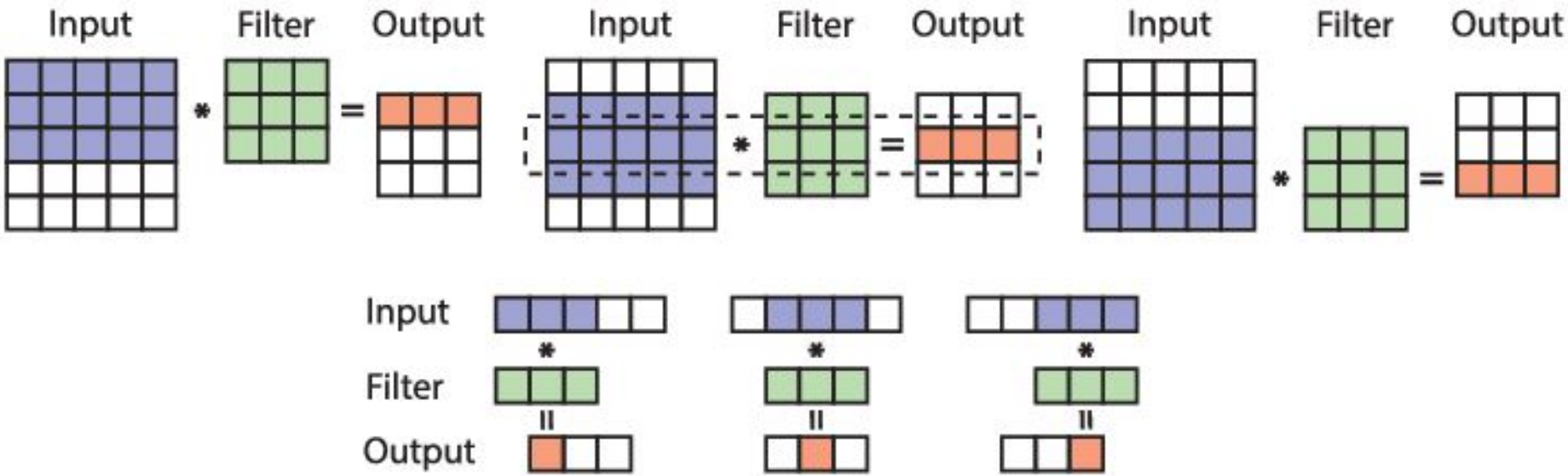
# Cross-Correlation Used in CNNs

- In true convolution the kernel is flipped before sliding over the input, whereas in cross-correlation the kernel is applied directly without flipping which is why CNNs actually use cross-correlation.
- Learning can adapt the kernel weights automatically, flipping the kernel provides no additional benefit, so CNNs use cross-correlation for simplicity and efficiency.

# Convolution Operation

Element wise matrix multiplication




Input
Filter
Output
Input
Filter
Output
Input
Filter
Output
Input
Filter
Output

Step-1

| | | | |
|---|---|---|---|
| 1 | 0 | -2 | 1 |
| -1 | 0 | 1 | 2 |
| 0 | 2 | 1 | 0 |
| 1 | 0 | 0 | 1 |

$\otimes$

| | |
|---|---|
| 0 | 1 |
| -1 | 2 |

$\rightarrow$

| | | |
|---|---|---|
| 1 | | |
| | | |
| | | |

Step-2

| | | | |
|---|---|---|---|
| 1 | 0 | -2 | 1 |
| -1 | 0 | 1 | 2 |
| 0 | 2 | 1 | 0 |
| 1 | 0 | 0 | 1 |

$\otimes$

| | |
|---|---|
| 0 | 1 |
| -1 | 2 |

$\rightarrow$

| | | |
|---|---|---|
| 1 | 0 | |
| | | |
| | | |

Step-3

| | | | |
|---|---|---|---|
| 1 | 0 | -2 | 1 |
| -1 | 0 | 1 | 2 |
| 0 | 2 | 1 | 0 |
| 1 | 0 | 0 | 1 |

$\otimes$

| | |
|---|---|
| 0 | 1 |
| -1 | 2 |

$\rightarrow$

| | | |
|---|---|---|
| 1 | 0 | 4 |
| | | |
| | | |

Step-4

| | | | |
|---|---|---|---|
| 1 | 0 | -2 | 1 |
| -1 | 0 | 1 | 2 |
| 0 | 2 | 1 | 0 |
| 1 | 0 | 0 | 1 |

$\otimes$

| | |
|---|---|
| 0 | 1 |
| -1 | 2 |

$\rightarrow$

| | | |
|---|---|---|
| 1 | 0 | 4 |
| 4 | | |
| | | |

Step-5

| | | | |
|---|---|---|---|
| 1 | 0 | -2 | 1 |
| -1 | 0 | 1 | 2 |
| 0 | 2 | 1 | 0 |
| 1 | 0 | 0 | 1 |

$\otimes$

| | |
|---|---|
| 0 | 1 |
| -1 | 2 |

$\rightarrow$

| | | |
|---|---|---|
| 1 | 0 | 4 |
| 4 | 1 | |
| | | |

Convolution converts a small local pattern match into one output value, and repeating this across the image builds a feature map.

Steps are:

1. Place kernel on image
2. Element-wise multiplication
3. Sum → one output pixel
4. Slide kernel

# Key Properties of Convolution

1. Local Receptive Fields: Each neuron sees only a small region
2. Parameter Sharing:
   a. Same kernel applied everywhere
   b. Reduces parameters drastically
3. Translation Equivariance: $f(T(x))=T(f(x))$
   a. Shift input → shift output

# Output Size Computation

$$\text{Output size} = \left\lfloor \frac{N - F + 2P}{S} \right\rfloor + 1$$

N: input size
F: filter size
P: padding
S: stride

**Example**

- Input: $32 \times 32$
- Filter: $5 \times 5$
- Stride: 1
- Padding: 0

$$(32 - 5 + 1) = 28 \Rightarrow 28 \times 28$$

# Padding and Stride

<u>Padding</u>

Valid: no padding

Same: output size = input size

<u>Stride</u>

Controls kernel movement

Larger stride → smaller output

$$P = \frac{F - 1}{2}$$

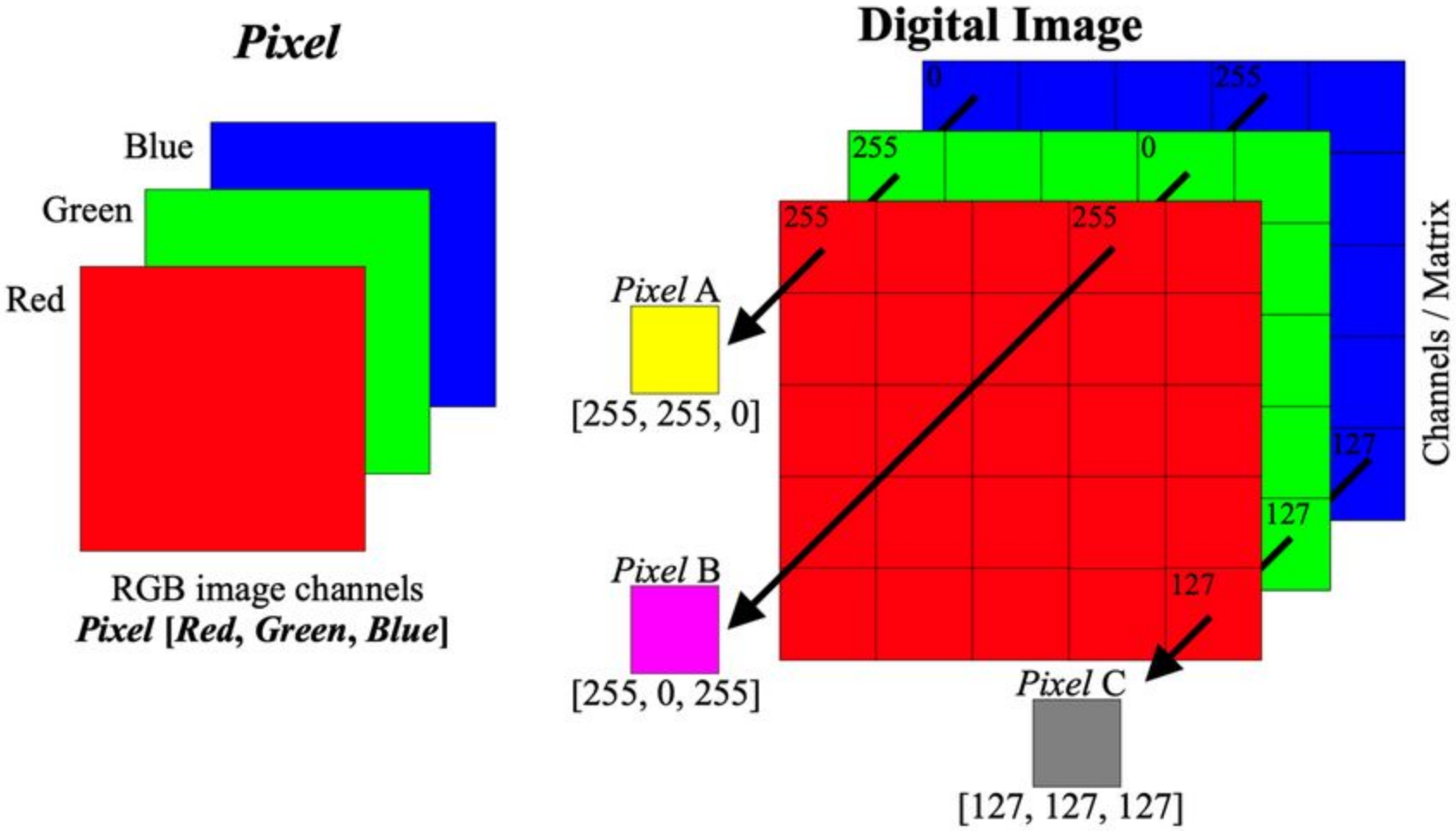
# Multi-Channel Convolution

Input: $H \times W \times C$

Kernel: $F \times F \times C$

$$Y(i,j) = \sum_{c=1}^{C} \sum_{m,n} X_c(i+m, j+n) K_c(m,n)$$

Explain RGB images clearly.


Pixel
Blue
Green
Red
RGB image channels
Pixel [Red, Green, Blue]
Digital Image
0
255
255
0
255
255
127
127
127
Channels / Matrix
Pixel A
[255, 255, 0]
Pixel B
[255, 0, 255]
Pixel C
[127, 127, 127]

# Parameter Count

For one filter:

$$\text{Params} = F \times F \times C + 1$$

For $K$ filters:

$$K(F \times F \times C + 1)$$

Contrast with fully connected layers.

# Parameter Count

| Aspect | Convolution | Fully Connected |
|---|---|---|
| Parameters | Few | Many |
| Spatial info | Preserved | Lost |
| Translation | Equivariant | No |

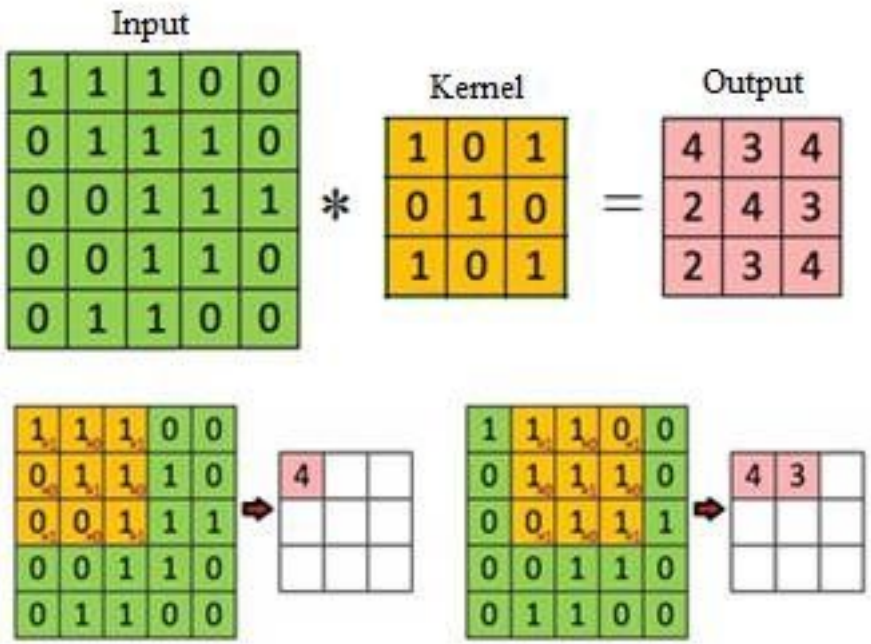
# Sparse Interaction

- Sparse interactions within convolution optimize computation by focusing on essential connections, reducing computational complexity.
- Sparse convolutions efficiently capture meaningful patterns while discarding unnecessary computations.
- In image recognition, sparse interactions enable the model to focus on relevant regions, enhancing efficiency.
- In Convolutional Neural Networks (CNNs), sparse interaction means:
  - Each output neuron depends only on a small region of the input, not the entire input.
  - This small region is called the receptive field.

- In a fully connected layer:
  - Every output neuron connects to all input pixels
  - That is dense connectivity
- In convolution:
  - A filter (kernel) of size F×F connects only to a small patch
  - Most input pixels are not connected to a given output neuron
  - Hence → Sparse interactions

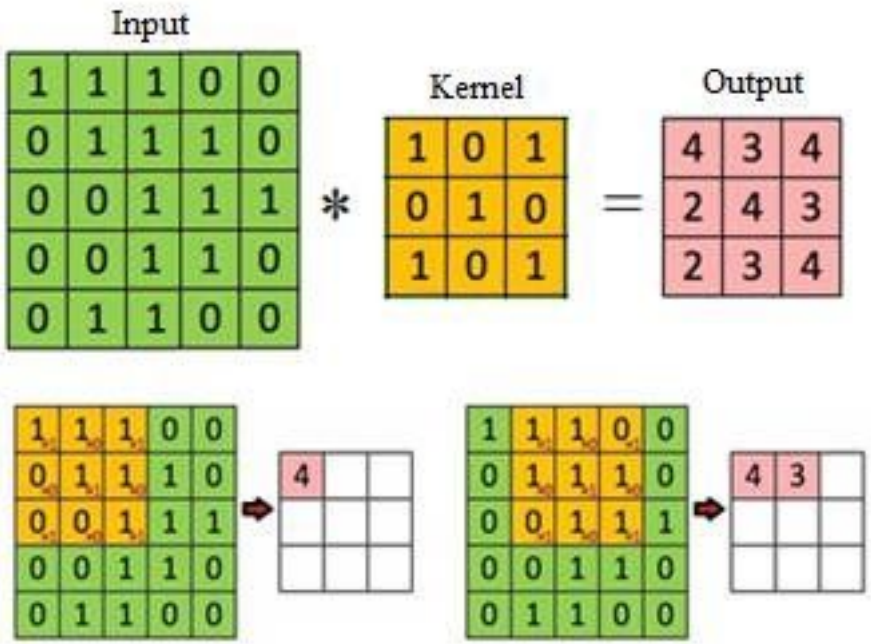
1. Place a 3×3 kernel on the top-left corner.
2. Multiply element-wise.
3. Sum → produces 1 output pixel.
4. Slide the kernel.
5. Repeat.

Each output pixel depends only on 9 input pixels, not 25.



1. Place a 3×3 kernel on the top-left corner.
2. Multiply element-wise.
3. Sum → produces 1 output pixel.
4. Slide the kernel.
5. Repeat.

Each output pixel depends only on 9 input pixels, not 25.



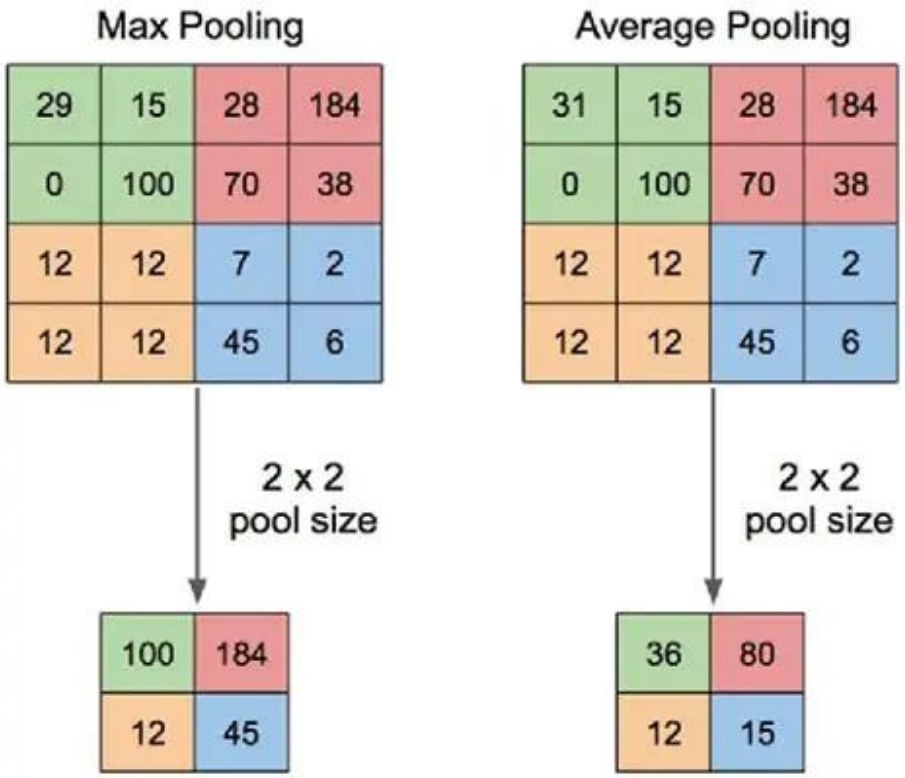
# Advantages

1. Reduces Parameters: Less memory, less overfitting
2. Efficient Computation: Smaller number of multiplications → Faster training.
3. Exploits Local Structure: Local edges, Corners, Textures. Nearby pixels are more related than distant pixels. Sparse interaction captures local patterns first.

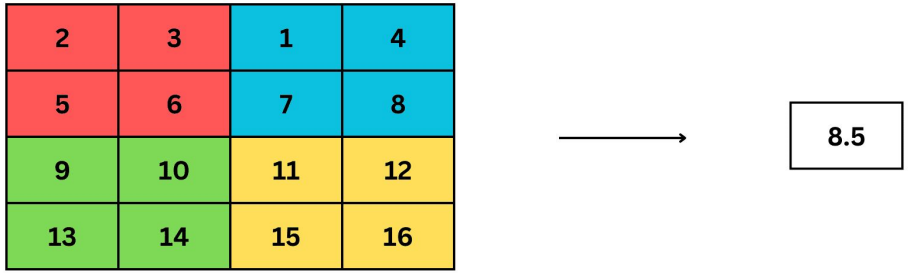
# Pooling, Convolutional Neural Networks, Parameters

# Pooling

- Pooling is a downsampling operation that reduces spatial dimensions of feature maps.
- It keeps important information while reducing size.
- Types of Pooling
  - Max Pooling: Takes maximum value in region.
  - Average Pooling: Takes average value.
  - Global Average Pooling: Averages entire feature map.
- Pooling reduces computation, reduces parameters, provides translation invariance, controls overfitting




# Global Average Pooling



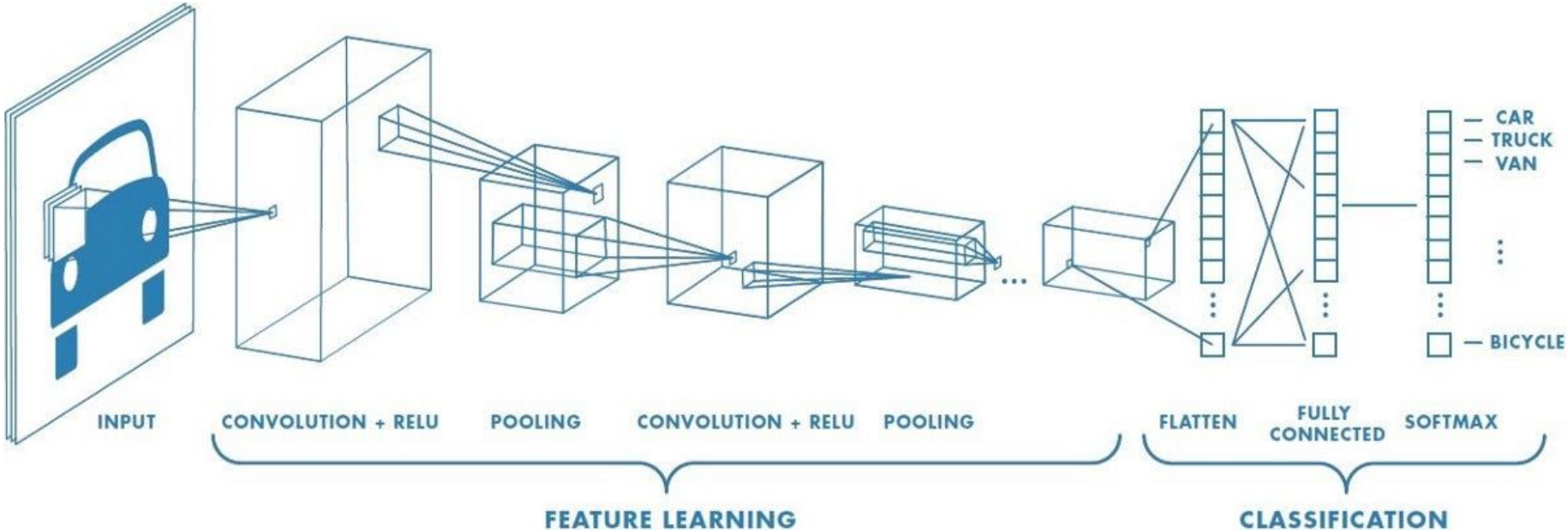

*No Conecpt of Windows. Take Average of all Values in the Matrix.*

- Pooling has:
  - No learnable parameters
  - Only hyperparameters (window size, stride)

# CNN

A CNN is a neural network designed for **grid-like data** (images).

Classic Architecture:



# Components of CNN

- **Convolution Layer**
  - Extracts features (edges, textures)
- **Activation (ReLU)**
  - $f(x)=\max(0,x)$
- **Pooling**
  - Downsamples
- **Fully Connected Layer**
  - Performs classification

- Hierarchical Feature Learning
  - Layer 1 → edges
  - Layer 2 → textures
  - Layer 3 → object parts
  - Deeper layers → objects
- This is representation learning.

# Why CNNs Work Well for Images

- Sparse interactions
- Parameter sharing
- Translation equivariance

# PARAMETERS IN CNNs

- Parameters in Fully Connected Layer
  - If input = 1000 neurons
  - Output = 500 neurons
  - Parameters: $1000 \times 500 = 500{,}000$ - Huge!
- Parameters in Convolution Layer
  - Suppose: Input channels = 3 (RGB), Filter size = $3 \times 3$, Number of filters = 10
  - Parameters per filter: $3 \times 3 \times 3 = 27$
  - Total parameters: $27 \times 10 = 270$ - Massive reduction

# General Formula

If: Kernel = $F \times F$, Input channels = $C$, Number of filters = $K$

Total parameters: $(F \times F \times C) \times K + K$ (+K for bias)

# Effect of Pooling on Parameters

- Pooling:
- Reduces spatial size
- Reduces number of activations
- Reduces parameters in FC layer
- Example:
- Without pooling: Feature map = 32×32×10
  - FC layer size = 10240 inputs
- With pooling: Feature map = 16×16×10
  - FC input = 2560
- Huge reduction.

# Variants of basic convolution function- dilated convolution, Transpose Convolution, 1x1 convolution, 3D Convolution

# Dilated Convolution

- Motivation
  - Problem: Small kernels → small receptive field
  - Deep stacking increases parameters (stacking multiple layers one after another to build a deeper neural network. Instead of using one large layer, we use many smaller layers stacked sequentially).
- Solution:
  - Increase receptive field without increasing parameters

# Dilated Convolution

- Definition: Dilated Convolution is a convolution technique that expands the kernel by inserting gaps (holes) between its elements.
- Also called: Atrous Convolution
- It is normal convolution with pixel skipping.

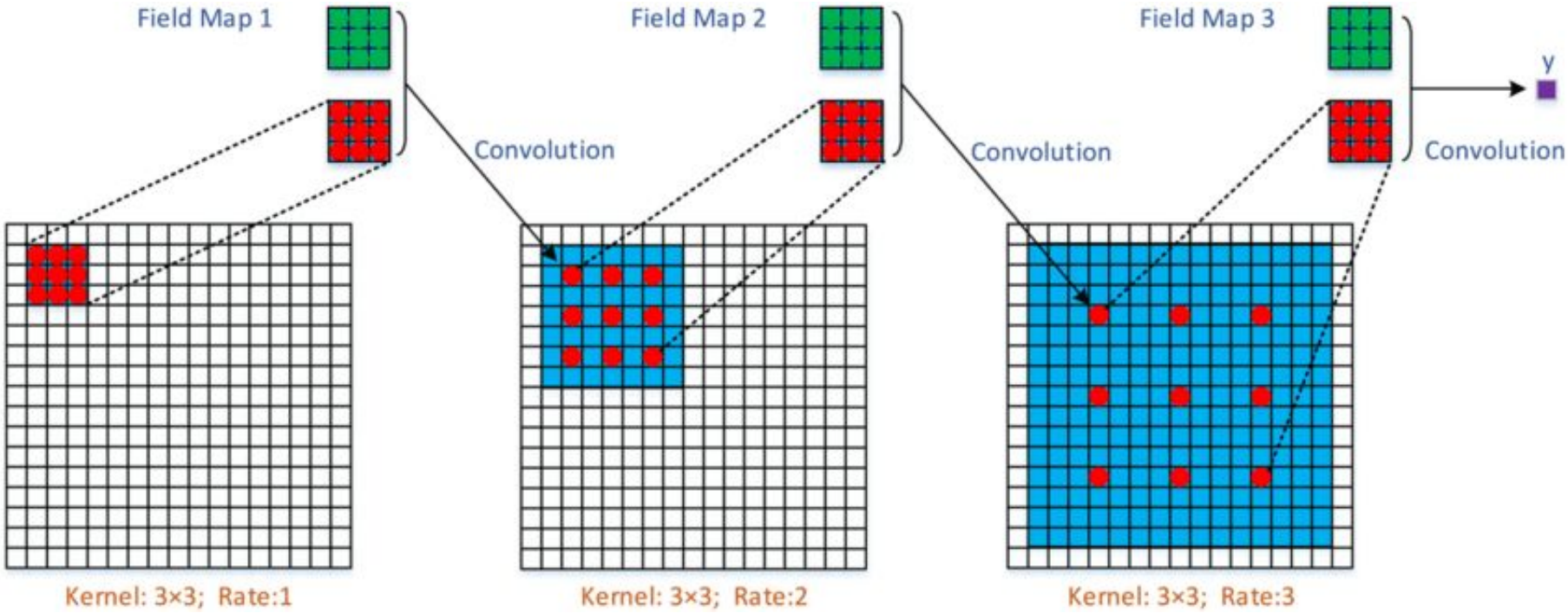
# Regular Convolution Review

- In standard convolution:
  - Filter of size k×k
  - Slides over input
  - Multiplies and sums values
  - Produces output feature map
- Receptive field depends on:
  - Kernel size
  - Stride

# Dilated Convolution Idea

- Instead of using a larger kernel like 5×5,
- We use a 3×3 kernel but insert gaps between elements.
- This increases coverage without increasing parameters.

- Rate = 1 → Normal convolution
- Dilation Rate = 2 → One gap between kernel elements (1 pixel skipped)
- Dilation Rate = 3 → Two gaps between elements (2 pixel skipped)



- Dilation rate is a hyperparameter.

# Effective Receptive Field

- Effective kernel size: $k_{effective}=k+(k-1)(d-1)$
- Example: For 3×3 kernel, d=2:
  - $k_{effective}=3+2(1)=5$
- So a 3×3 kernel behaves like 5×5 coverage.
- But parameters remain: 3×3=9
  - Because:
    - Number of weights remains same
    - Only spacing changes
    - No new learnable weights added
- Hence: Larger receptive field at same parameter cost

# Applications

- Semantic Segmentation: Need large spatial context to classify each pixel. Used in: Google DeepLab models.
- Audio Processing: Captures long temporal dependencies.
- Time-Series Modeling: Learns long-range patterns efficiently.

# Advantages

- Larger receptive field
- Same number of parameters
- No extra computational burden
- Good for dense prediction tasks

# Possible Issue

- High dilation rates may cause: Gridding artifacts (Information sampled in a checkerboard pattern).

# Transpose Convolution
## (Deconvolution / Fractionally Strided Convolution)

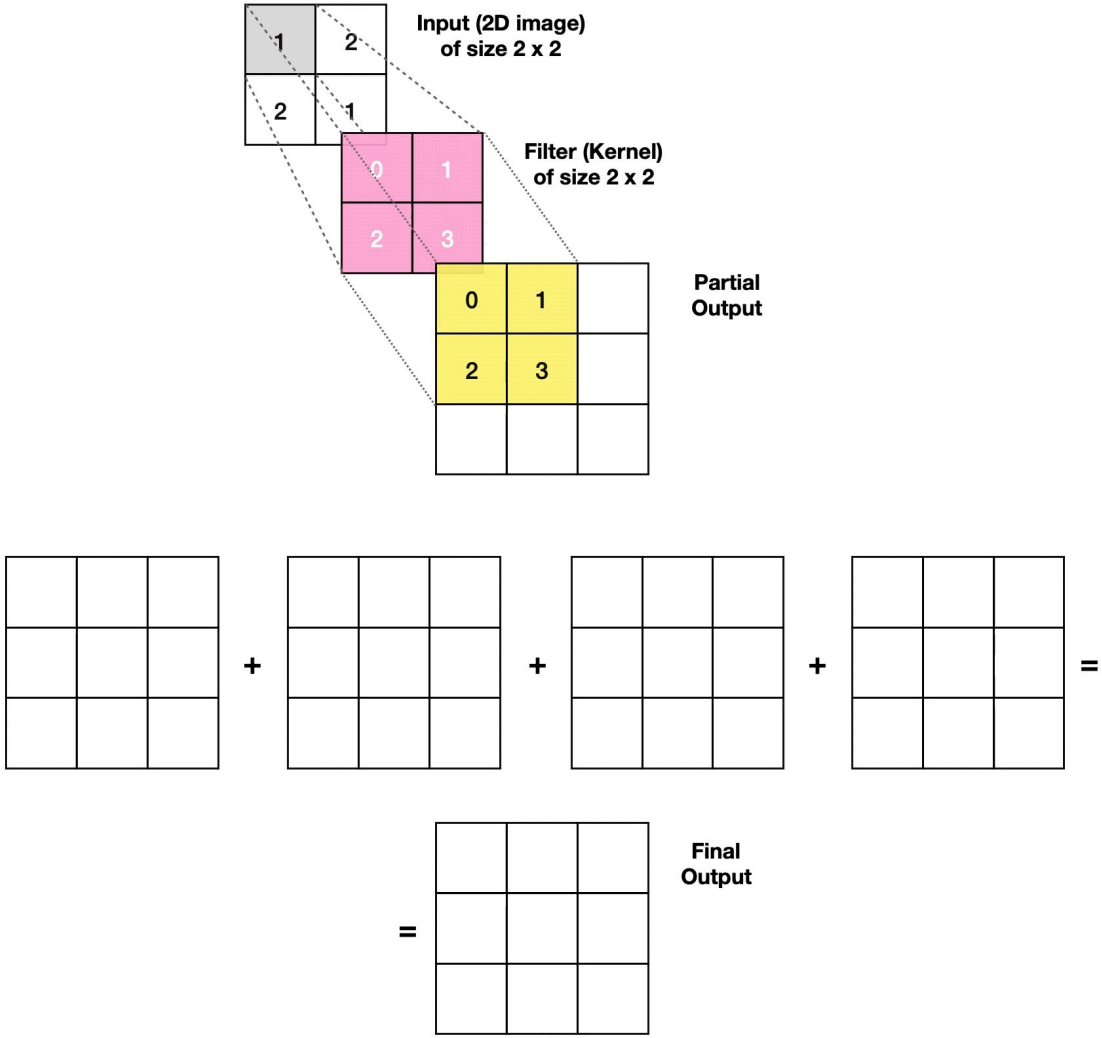
- Problem
  - Standard convolution reduces spatial size
  - Pooling reduces size further
- But in tasks like: Semantic segmentation, Image generation, Autoencoders
- We need to: Increase spatial resolution (Upsampling)

- Definition: Transpose Convolution is a convolution operation used to increase the spatial dimensions of feature maps.
- It is:
  - Not a true inverse of convolution
  - A learnable upsampling operation
- Also called: Deconvolution (informal name)

- In matrix form:
  - Convolution can be written as matrix multiplication
  - Transpose convolution uses the transpose of that matrix
- Hence the name: Transpose Convolution

- Regular Convolution: Input→SmallerOutput
- Transpose Convolution: Input→LargerOutput
- It performs:
  - Insert zeros between input pixels
  - Apply convolution
  - Produce larger output

stride=1


Input (2D image)
of size 2 x 2
1
2
2
1
Filter (Kernel)
of size 2 x 2
0
1
2
3
Partial
Output
0
1
2
3
+
+
+
=
=
Final
Output

- For stride = 2:
  - Zeros inserted between pixels
  - Convolution spreads values
  - Output dimension increases

# Output Size Formula

- Without padding: Output=(Input−1)×Stride+Kernel
- With padding: Output=(Input−1)×Stride−2P+Kernel
- Where:
  - P = padding
  - Stride = stride
  - Kernel = filter size

Input size = 4

Kernel = 3

Stride = 2

Padding = 0

$$Output = (4 - 1) \times 2 + 3 = 9$$

So output size = 9

# Applications

- Semantic Segmentation: Used in U-Net
- Image Generation: Used in Generative Adversarial Nets
- Reconstruct high-resolution images.

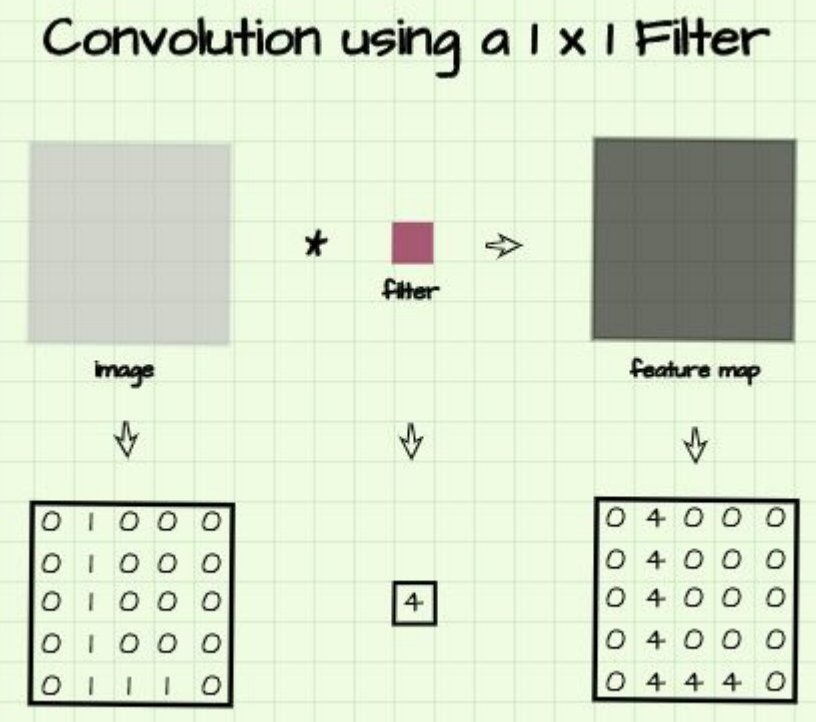
- Advantages
  - Learnable upsampling
  - Better than simple interpolation
  - Captures spatial patterns while increasing size
- Problem
  - Checkerboard artifacts
    - Cause: Uneven overlap during upsampling
  - Solution:
    - Use smaller kernels
    - Use upsampling + convolution separately

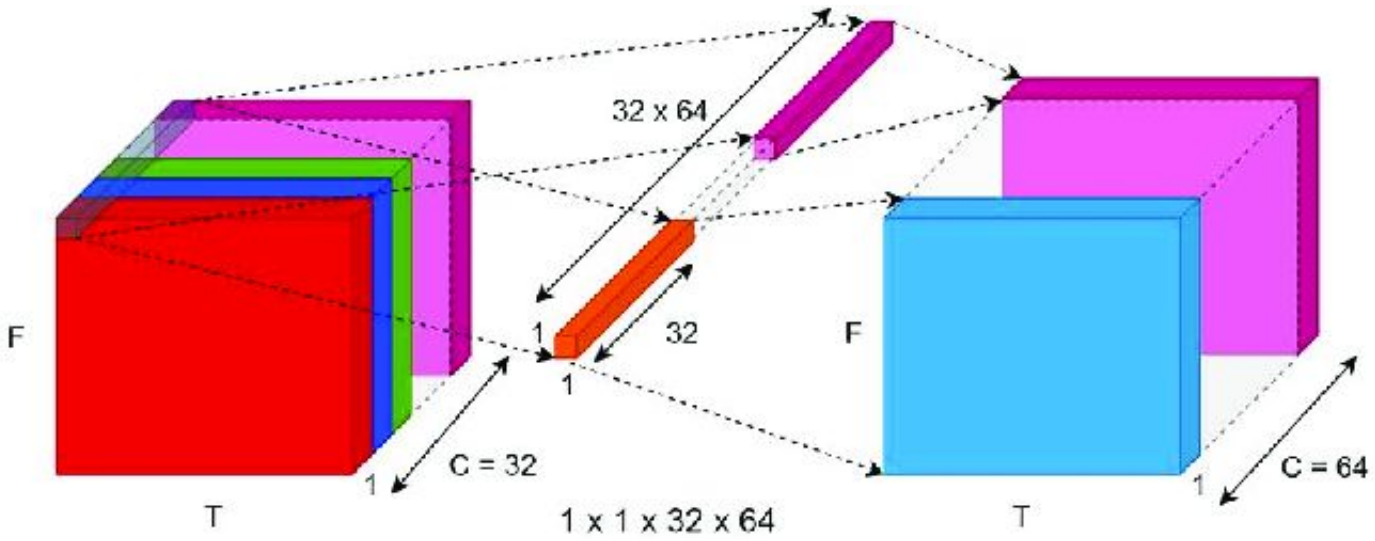
# 1×1 Convolution (Pointwise Convolution)

- It does not capture spatial information, but it learns across channels.
- A convolution where: Kernel=1×1
- It operates at each spatial location but combines information across channels.

# Input–Output Shape

- If input is: H×W×C
- Using K filters of size 1×1:
- Output becomes: H×W×K
- Spatial size unchanged
- Channel dimension changes






c) pointwise/1-by-1 convolution layer

- Each pixel:
  - Takes all C channel values
  - Multiplies with C weights
  - Produces one new value
- It is like a fully connected layer applied at every pixel location.

# Parameter Count

- For:
  - Input channels = C
  - Number of filters = K
- Parameters: $1 \times 1 \times C \times K + K$
- Much smaller than $3 \times 3$ convolution.

# Why Important?

- Channel mixing
- Dimensionality reduction
- Computational efficiency
- Adds non-linearity

Used heavily in:

  - Inception
  - ResNet

# Applications

- Bottleneck layers
- Feature compression
- Attention mechanisms

# 3D Convolution

- 2D Convolution works on: Height×Width
- But what about:
  - Videos (time dimension)?
  - Medical volumes (CT/MRI)?
  - fMRI data?
- We need 3D convolution.

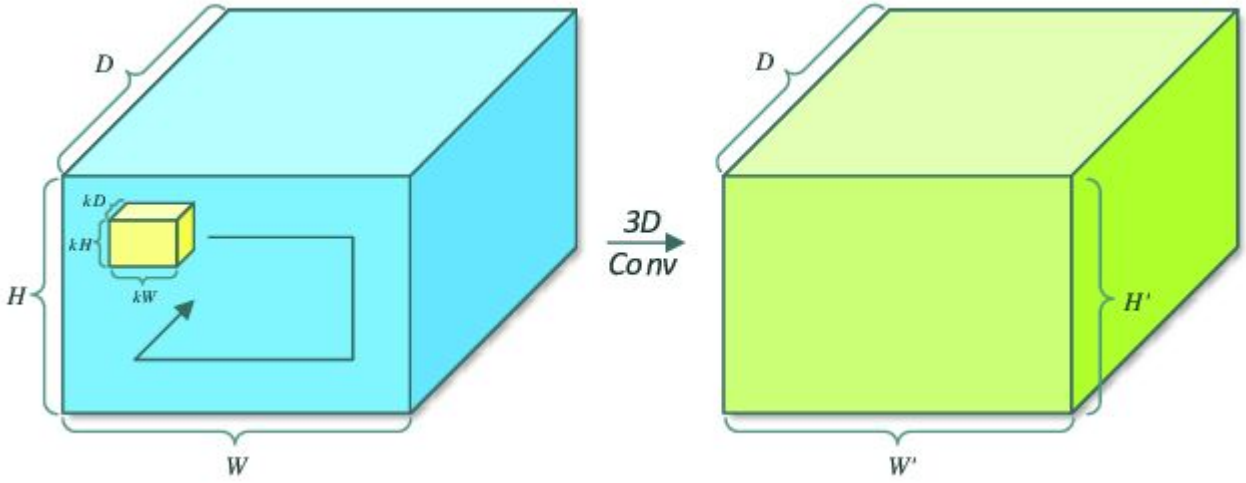
# What is 3D Convolution?

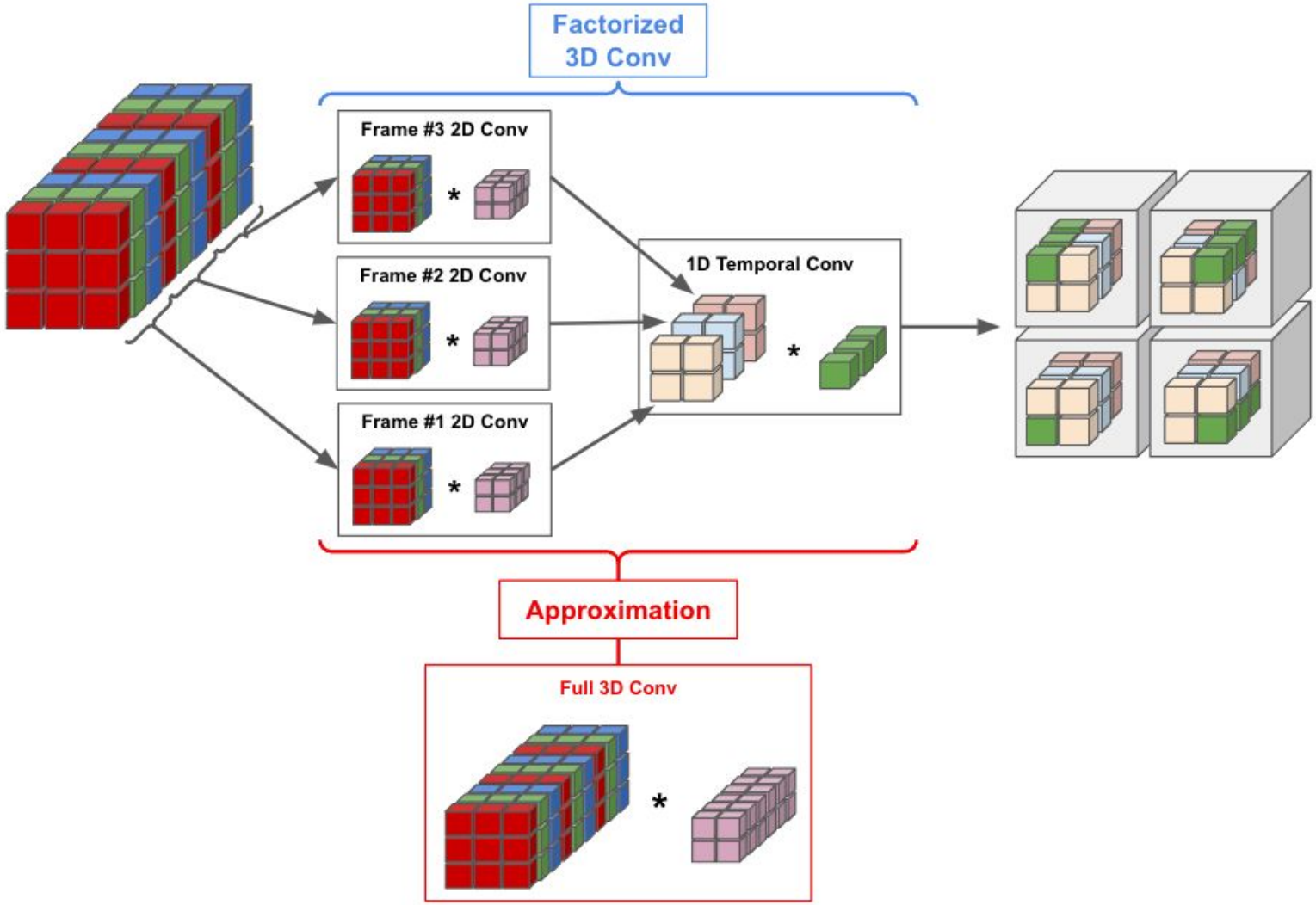
- 3D Convolution uses a kernel of size: F×F×F
- It moves across:
  - Height
  - Width
  - Depth (time or volume)

# Input–Output Shape

- If input is: $D \times H \times W \times C$
- Kernel: $F \times F \times F$
- Output preserves depth relationships.

Instead of sliding over 2D image, The kernel slides over a 3D block.




Factorized
3D Conv
Frame #3 2D Conv
Frame #2 2D Conv
Frame #1 2D Conv
1D Temporal Conv
Approximation
Full 3D Conv

# Parameter Count

If:

Kernel = $F \times F \times F$

Input channels = $C$

Number of filters = $K$

Parameters: $F \times F \times F \times C \times K + K$

Parameters increase rapidly.

# Applications

- Video classification
- Action recognition
- MRI / CT analysis

- Example: C3D

# Advantages & Challenges

- Captures temporal/volumetric patterns

Challenges

- High memory usage
- High computational cost

# Comparison of All Variants

| Type | Kernel Size | Spatial Size Change | Receptive Field | Main Purpose |
|---|---|---|---|---|
| Regular Conv | F×F | Usually ↓ | Normal | Feature extraction |
| Dilated Conv | F×F (with gaps) | Same | Larger | Context capture |
| Transpose Conv | F×F | ↑ | Same | Upsampling |
| 1×1 Conv | 1×1 | Same | Same | Channel mixing |
| 3D Conv | F×F×F | Depends | Larger (depth included) | Video/Volume |

## Conceptual Comparison

- Dilated → increases receptive field
- Transpose → increases spatial resolution
- 1×1 → increases channel interaction
- 3D → increases dimensional coverage

# Backpropagation in Convolutional Layers

- Backpropagation, short for "backward propagation of errors," is an algorithm used to calculate the gradient of the loss function of a neural network with respect to its weights.
- It is essentially a method to update the weights to minimize the loss.
- Backpropagation is crucial because it tells us how to change the weights to improve the network’s performance.

# Role of Backpropagation in CNNs

- In a CNN, backpropagation plays a crucial role in fine-tuning the filters and weights during training, allowing the network to better differentiate features in the input data.
- CNNs typically consist of multiple layers, including convolutional layers, pooling layers, and fully connected layers.
- Each of these layers has weights and biases that are adjusted via backpropagation.

# Fundamentals of Backpropagation

- Backpropagation, in essence, is an application of the chain rule from calculus used to compute the gradients (partial derivatives) of a loss function with respect to the weights of the network.
- The process involves three main steps:
  - the forward pass
  - loss calculation
  - the backward pass

# The Forward Pass

- During the forward pass, input data (e.g., an image) is passed through the network to compute the output.
- For a CNN, this involves several key operations:
  1. Convolutional Layers: Each convolutional layer applies numerous filters to the input.
     - For a given layer $l$ with filters denoted by $F$, input $I$, and bias $b$, the output $O$ is given by: $O=(I*F)+b$
     - Here, $*$ denotes the convolution operation.

2. Activation Functions: After convolution, an activation function $\sigma$ (e.g., ReLU) is applied element-wise to introduce non-linearity: $O=\sigma((I*F)+b)$
3. Pooling Layers: Pooling (e.g., max pooling) reduces dimensionality, summarizing the features extracted by the convolutional layers.

# Loss Calculation

- After computing the output, a loss function L is calculated to assess the error in prediction.
- Common loss functions include mean squared error for regression tasks or cross-entropy loss for classification:

$$L = -\sum y log(\hat{y})$$

- Here,
  - y is the true label
  - ŷ is the predicted label.

# The Backward Pass (Backpropagation)

The backward pass computes the gradient of the loss function with respect to each weight in the network by applying the chain rule:

1. **Gradient with respect to output:**
   a. First, calculate the gradient of the loss function with respect to the output of the final layer:

$$\frac{\partial L}{\partial O}$$

2. **Gradient through activation function:** Apply the chain rule through the activation function:

$$\frac{\partial L}{\partial I} = \frac{\partial L}{\partial O}\frac{\partial O}{\partial I}$$

For ReLU, $\frac{\partial O}{\partial I} = 1$ for I>0 and 0 otherwise

3. Gradient with respect to filters in convolutional layers:
   a. Continue applying the chain rule to find the gradients with respect to the filters:

$$\frac{\partial L}{\partial F} = \frac{\partial L}{\partial O} * rot180(I)$$

rot180(I): rotates the input by 180 degrees, aligning it for the convolution operation used to calculate the gradient with respect to the filter.

# Weight Update

- Using the gradients calculated, the weights are updated using an optimization algorithm such as SGD:

$$F_{new} = F_{old} - \eta \frac{\partial L}{\partial F}$$

η is the learning rate, which controls the step size during the weight update.

# Challenges in Backpropagation

- **Vanishing Gradients:** Gradients become too small to make significant changes in weights, stalling the training.
    - Advanced activation functions like ReLU and optimization techniques such as batch normalization are used to mitigate this issue.
- **Exploding Gradients:** Gradients can become excessively large; this is known as exploding gradients.
    - Controlled by techniques such as gradient clipping.

# Cross Entropy Loss Function

- In classification problems, a machine learning model predicts the probability of each class for any given input.
    - Each data point truly belongs to only one class (probability 1 for one class, 0 for others).
- Cross-entropy loss is a way to measure how close a model's predictions are to the correct answers in classification problems.
- It helps train models by rewarding correct answers and penalizing wrong ones making it a key part of building reliable machine learning classifiers.

# Types of Cross-Entropy Loss Function

1. Binary Cross Entropy Loss

- Binary cross-entropy (log loss) is a loss function used in binary classification problems.
- It quantifies the difference between the actual class labels (0 or 1) and the predicted probabilities output by the model.
- The lower the binary cross-entropy value, the better the model's predictions align with the true labels.

- Binary Cross-Entropy (BCE) is defined as:

$$BCE = -\frac{1}{N}\sum_{i=1}^{N}[y_i log(p_i) + (1 - y_i) log(1 - p_i)]$$

Where

- N is the number of observations
- $y_i$ is the actual binary label (0 or 1) of the $i^{th}$ observation.
- $p_i$ is the predicted probability of the $i^{th}$ observation being in class 1.

Since the model's output is a probability between 0 and 1, minimizing binary cross-entropy during training helps improve predictive accuracy, ensuring the model effectively distinguishes between two classes.

# Working

- Binary Cross-Entropy measures the distance between the true labels and the predicted probabilities.
- When the predicted probability $p_i$ is close to the actual label $y_i$, the BCE value is low, indicating a good prediction.
- If the predicted probability deviates significantly from the actual label, BCE value is high, indicating a poor prediction.
- The logarithmic component of the BCE function penalizes wrong predictions more heavily than correct ones.

- Ex:
  - if the true label is 1 and the predicted probability is close to 0, the loss is substantial.
- This characteristic makes BCE particularly effective in driving the model to improve its predictions during training.

# Importance of Binary Cross Entropy

1. Training Deep Learning Models: Used as the loss function for training neural networks in binary classification tasks. Helps adjust the model's weights to minimize the prediction error.
2. Probabilistic Interpretation: Makes it suitable for applications where understanding the confidence of predictions is important, such as in medical diagnosis or fraud detection.
3. Model Evaluation: Clear and interpretable metric for evaluating the performance of binary classification models. Lower BCE values indicate better model performance.

4. Handling Imbalanced Data: BCE can be particularly useful in scenarios with imbalanced datasets, where one class is significantly more frequent than the other. By focusing on probability predictions, it helps the model learn to make accurate predictions even in the presence of class imbalance.

# Ex:

| Observation | True Label ($y$) | Predicted Probability ($p$) |
|---|---|---|
| **1** | 1 | 0.9 |
| **2** | 0 | 0.2 |
| **3** | 1 | 0.8 |
| **4** | 0 | 0.4 |

Observation 1: Here, True label $y_1$=1 and Predicted probability $p_1$=0.1

$$Loss_1 = -(1.log(0.9)) + (1-1).log(1-0.9) = -log(0.9)$$

$$\approx -(-0.1054) = 0.1054$$ (machine learning uses natural log)

Similarly

Predicted probability $p_2$ =0.2 and $Loss_2$=0.223

Predicted probability $p_3$ =0.8 and $Loss_3$=0.2231

Predicted probability $p_4$ =0.4 and $Loss_4$=0.5108

Next, we sum the individual losses and calculate the average:

Total Loss=0.1054+0.2231+0.2231+0.5108=1.0624

$$AverageLoss(BCE) = \frac{1.0624}{4} = 0.2656$$

Therefore, the Binary Cross-Entropy loss for these observations is approximately 0.2656.

2. Multiclass Cross Entropy Loss (categorical cross-entropy or softmax loss)

- Multiclass Cross-Entropy Loss, is a widely used loss function for training models in multiclass classification problems.
- For a dataset with N instances, Multiclass Cross-Entropy Loss is calculated as

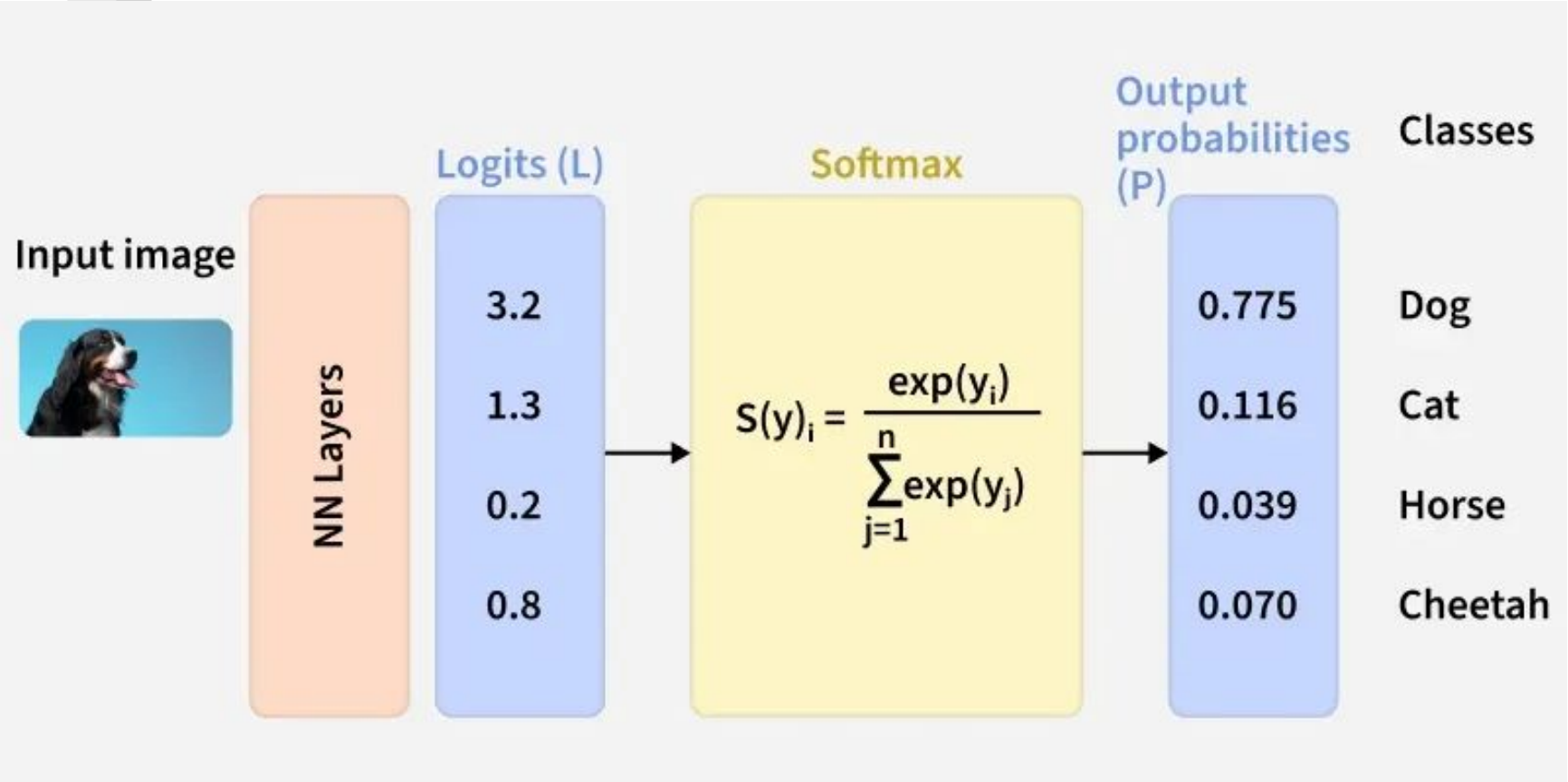
$$CE = -\frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{C} y_{i,j}.\,log(p_{i,j})$$

- Where:
  - N is number of samples,
  - C is the number of classes.
  - $y_{ij}$ is 1 if class j is correct for sample i, 0 otherwise.
  - $p_{ij}$ is model-predicted probability of sample i being in class j.

# Categorical Cross-Entropy in Multi-Class Classification

- It measures the difference between the predicted probability distribution and the true one-hot encoded labels, guiding the model to assign higher probabilities to the correct class.
- It is used when there are more than two classes.
- Works with softmax outputs where probabilities sum to 1.
- Higher loss means the prediction is far from the true class, lower loss means the model is performing well.
- Commonly used in image classification, text classification and speech recognition tasks.

**Neural networks are converted into Softmax probabilities and used in Categorical Cross-Entropy (CCE) to compute loss for the true class.**



# Working

- Categorical Cross-Entropy measures the difference between the true labels and the predicted probabilities of a model.
- It penalizes the model when it assigns low confidence to the correct class.
- Formula is: $L(y, \hat{y}) = -\sum_{i=1}^{C} y_i log(\hat{y}_i)$
- where:
  - $L(y,\hat{y})$: categorical cross entropy loss
  - $y_i$: true label for class i
  - $\hat{y}_i$: predicted label for class i
  - C: number of classes

- *Prediction of Probabilities:* The model uses a Softmax layer to convert raw logits into probabilities for each class.
- *Comparison with True Class:* Predicted probabilities are matched with one-hot encoded labels to determine the correct class.
- *Calculation of Loss:* CCE calculates the negative log of the predicted probability for the true class, giving lower loss for higher confidence and higher penalty for low confidence.

| Parameters | Categorical Cross-Entropy | Binary Cross-Entropy |
|---|---|---|
| **Use Case** | Multi-class classification | Binary classification |
| **Label Format** | One-hot encoded vector | Single label |
| **Interpretation** | Penalizes wrong predictions across all classes | Penalizes wrong prediction for the single class |
| **Activation Function** | Softmax | Sigmoid |
| **Output** | Probability distribution across multiple classes | Single probability for positive class |

# Applications

- *Handwritten Digit Recognition:* Classifying digits 0 to 9 in apps like postal mail sorting.
- *Email Classification:* Categorizing emails into multiple folders like Inbox, Promotions Social, etc.
- *Sentiment Analysis:* Determining if a review is Positive, Negative or Neutral.
- *Medical Imaging:* Detecting types of diseases from X-rays or MRI scans.
- *Speech Recognition:* Recognizing different words or commands in voice assistants.

# Advantages

- *Effective for Multi-Class Problems:* Perfectly suited for tasks with more than two classes.
- *Probabilistic Interpretation:* Works naturally with Softmax outputs to produce meaningful probabilities.
- *Sensitive to Incorrect Predictions:* Penalizes wrong predictions more helping models learn better.
- *Smooth Gradient:* Provides continuous and differentiable loss ideal for gradient-based optimization.

# Limitations

- *Requires One-Hot Labels:* Needs proper encoding of true labels, not suitable for raw class integers.
- *Overconfidence Risk:* Models can become overconfident in predictions if not regularized.
- *Not for Multi-Label Problems:* Works for single-class predictions per sample, not multi-label classification.
- *Sensitive to Class Imbalance:* Can give biased training if classes are unevenly distributed.

# Softmax Loss Function

- Multiclass classification is an application of deep learning/machine learning where the model is given input and renders a categorical output corresponding to one of the labels that form the output.
- For example, providing a set of images of animals and classifying it among cats, dogs, horses, etc.
- For this purpose, where the model outputs multiple outputs for each class, a simple logistic function (or sigmoid function) cannot be used.
- Thus, another activation function called the Softmax function is used along with the cross-entropy loss.

- The softmax formula is represented as:

$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

- where the values of $z_i$ are the elements of the input vector and they can take any real value.
- The denominator of the formula is normalised term which guarantees that all the output values of the function will sum to 1, thus making it a valid probability distribution.

- The softmax function and the sigmoid function are similar to each other.
- Softmax operates on vector values while the sigmoid takes scalar values.
- Sigmoid function is a specific case of the softmax function and it is for a classifier with only two input classes.
- The logistic function, often known as the logistic sigmoid function, is commonly referred to as "sigmoid function" in the context of machine learning.

$$sigmoid(x) = \frac{1}{1 + e^{-x}}$$

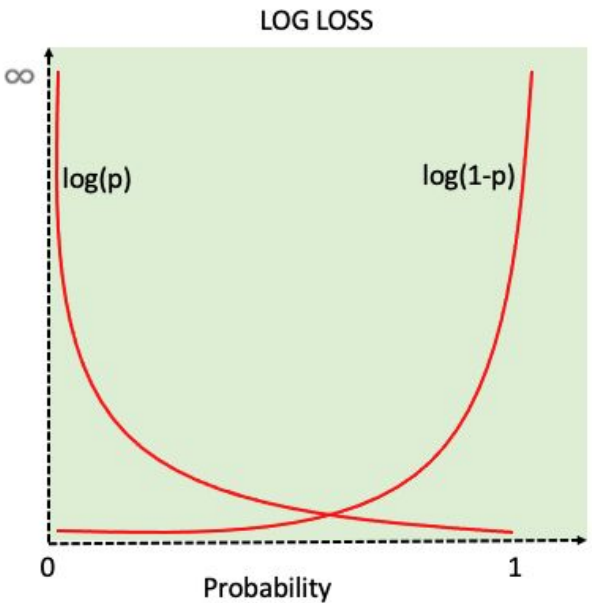
- Sigmoid function is used for classification between 2 classes, i.e., 1 and 0.
- In the case of Multiclass classification, the softmax function is used.
- The softmax converts the output for each class to a probability value (between 0-1), which is exponentially normalized among the classes.

# How to interpret Cross Entropy Loss?

- The cross-entropy loss is a scalar value that quantifies how far off the model's predictions are from the true labels.
- For each sample in the dataset, the cross-entropy loss reflects how well the model's prediction matches the true label.
- A lower loss for a sample indicates a more accurate prediction, while a higher loss suggests a larger discrepancy.

# Interpretability for Binary Classification:

- In binary classification, there are 2 classes.
- To interpret the loss value,
  - If the true label is 1, the loss is primarily influenced by how close the predicted probability for class 1 is to 1.0
  - If the true label is 0, the loss is influenced by how close the predicted probability for class 1 is to 0.0



# Interpretability for Multiclass Classification:

- In multiclass classification, only the true label contributes towards the loss as for other labels being zero does not add anything to the loss function.
- Lower loss indicates that the model is assigning high probabilities to the correct class and low probabilities to incorrect classes.

# Key features of Cross Entropy loss

- *Probabilistic Interpretation:* Guides models to output probabilities near the true class labels.
- *Differentiable:* Supports optimization via gradient descent.
- *Standard for Neural Networks:* Especially with softmax (multiclass) or sigmoid (binary) output layers.
- *Strong Penalization:* Assigns high penalty to confident but wrong predictions.
- *Library Support:* Implemented in all major ML libraries like PyTorch, TensorFlow, scikit-learn, etc.

# Alexnet

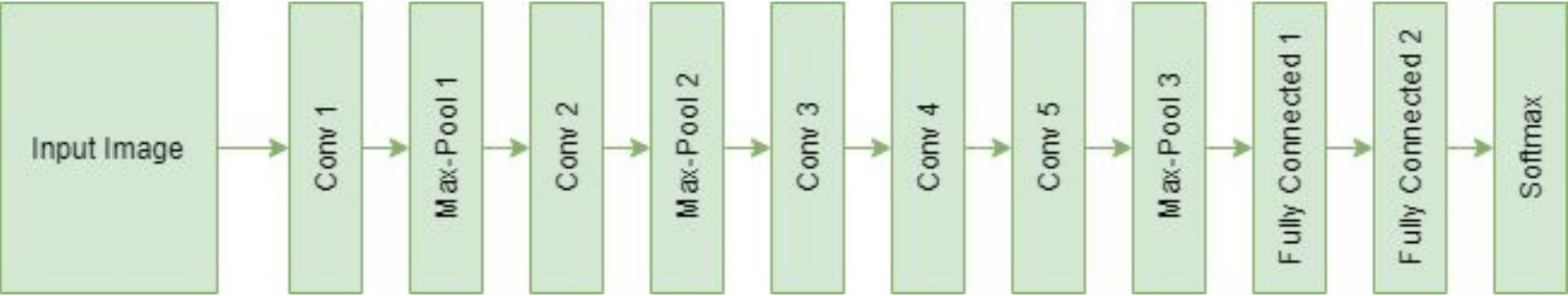
- AlexNet is a deep learning model that made a big impact in image recognition.
- It became famous for its ability to classify images accurately.
- It won the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) 2012 with a top-5 error rate of 15.3% (beating the runner up which had a top-5 error rate of 26.2%).

- Most important features of the AlexNet are:
  - *Overfitting Prevention:* Dropout (0.5) was applied to the first two fully connected layers and data augmentation dynamically expanded the dataset hence both helping in reducing overfitting.
  - *Faster Training:* ReLU activation was used instead of tanh or sigmoid, leading to a 6× speedup in training by avoiding activation saturation.

# AlexNet Architecture

- 5 convolutional layers with Max-Pooling applied after the 1st, 2nd and 5th layers to enhance feature extraction.
- Overlapping Max-Pooling uses a 3×3 filter with stride 2 which improved performance by reducing top-1 error by 0.4% and top-5 error by 0.3% compared to non-overlapping pooling.
- Followed by 2 fully connected layers each using dropout to prevent overfitting.
- Ends with a softmax layer for final classification.

# Architecture



# Advantages of AlexNet

- Use of ReLU Activation: First major architecture to use ReLU (Rectified Linear Unit). Enabled faster training.
- Dropout for Regularization: Reduce overfitting by randomly disabling neurons during training.
- GPU Utilization: Split the network across two GPUs, benefits from parallel computing for faster training.
- Overlapping Max-Pooling: Used overlapping pooling layers to improve generalization and reduce top-1 (highest classification prediction) and top-5 classification prediction errors.

# Disadvantages of AlexNet

- *Large Model Size:* around 60 million parameters (memory-intensive and slow for inference on low-resource devices).
- *High Computational Cost:* Training is computationally expensive even though it was optimized for GPUs.
- *Manual Architecture Design:* lacks modularity and automation, unlike modern approaches like NAS or EfficientNet.

- *Not Optimal for Small Datasets:* Tends to overfit on smaller datasets like CIFAR-10 or MNIST without heavy regularization.
- *Outdated Compared to Modern Architectures:* Lacks innovations like residual connections (ResNet), depthwise separable convolutions (MobileNet) and attention mechanisms (ViT).

# Applications

1. *Image Classification:* Originally built for classifying high-resolution images into 1000 object categories (ImageNet dataset).
2. *Feature Extraction:* Intermediate layers are often used as pretrained feature extractors for transfer learning tasks.
3. *Object Detection:* Forms the backbone in early detection systems like R-CNN when combined with region proposal methods.

4. *Medical Imaging:* Applied to classify abnormalities in X-rays, MRIs or retinal scans by fine-tuning on domain-specific datasets.
5. *Facial Recognition and Emotion Detection:* Can be adapted for face verification, expression analysis or identity recognition tasks.
6. *Autonomous Vehicles:* Used in early perception modules for identifying road signs, pedestrians or obstacles.

# GoogLeNet Architecture

- GoogLeNet (Inception V1) is a deep CNN architecture designed for efficient image classification.
- It introduces the Inception module, which performs multiple convolution operations (1x1, 3x3, 5x5) in parallel, along with max pooling and concatenates their outputs.
- The architecture is optimized for speed and performance, which makes it suitable for large-scale visual recognition tasks.
- Implements 1×1 convolutions, global average pooling and the Inception module, to improve depth and computational efficiency.

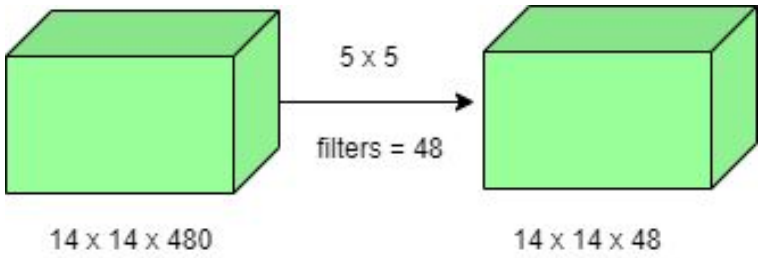
# Key Features

1. 1x1 Convolution:

- Employed primarily for dimensionality reduction.
- Helps to decrease the number of trainable parameters while enabling deeper and more efficient architectures.

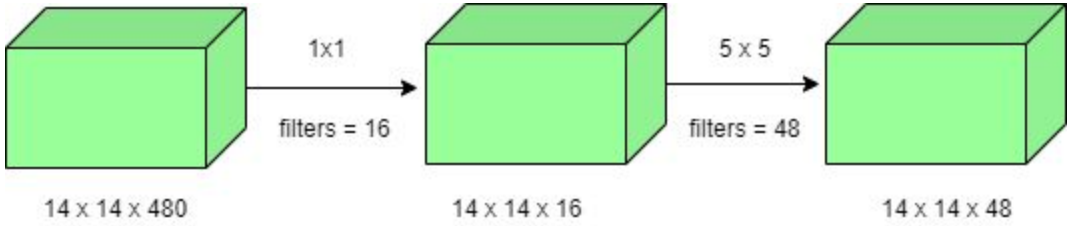
**Ex:** *Without using 1×1 convolution for dimensionality reduction, the 5×5 convolution requires about 112.9 million operations*

- **Without 1×1 Convolution:** (14 × 14 × 48) × (5 × 5 × 480) = 112.9M operations




- 5x5 size filter applied to 480 channels. And there are 48 filters.
- Operations for One Output Pixel=5x5x480=12000
- Each filter produces a feature map of size:14×14=196
    - 48 filters produce: 196x48=9408 output pixels
- Total operations=12000x9408=112,896,000≈112.9M

- **With 1×1 Convolution:** (14 × 14 × 16) × (1 × 1 × 480) + (14 × 14 × 48) × (5 × 5 × 16) = 5.3M operations




- Input size: 14×14×480, apply 16 filters of size 1×1×480.
- Each filter produces one output channel, so output is: 14×14×16
- Operations per output pixel: 1x1x480
- Number of output pixels: 14×14×16=3136
- Total operations in this step(14×14×16)×(1×1×480)=1,505,280≈ 1.5M operations.

- Input to the 5x5 convolution layer: 14×14×16
- Apply 48 filters of size: 5×5×16
- Output size becomes: 14×14×48
- Operations per output pixel: 5×5×16=400
- Number of output pixels: 14×14×48=9408
- Total operations: 9408×400=3763200 ≈ 3.8M operations
- Total Operations =1.5M+3.8M ≈5.3M
- 1×1 convolution reduces computation by about 21× while still allowing the network to learn useful features.

1×1 convolution acts like a channel compressor — it reduces depth before applying expensive filters like 5×5.

**2. Global Average Pooling:** In traditional architectures like AlexNet, fully connected layers at the end introduce a large number of parameters.

- GoogLeNet replaces these with Global Average Pooling, which computes the average of each feature map (e.g. converting 7×7 maps to 1×1), this significantly reduces the model's parameter count and solves overfitting.

Benefits:

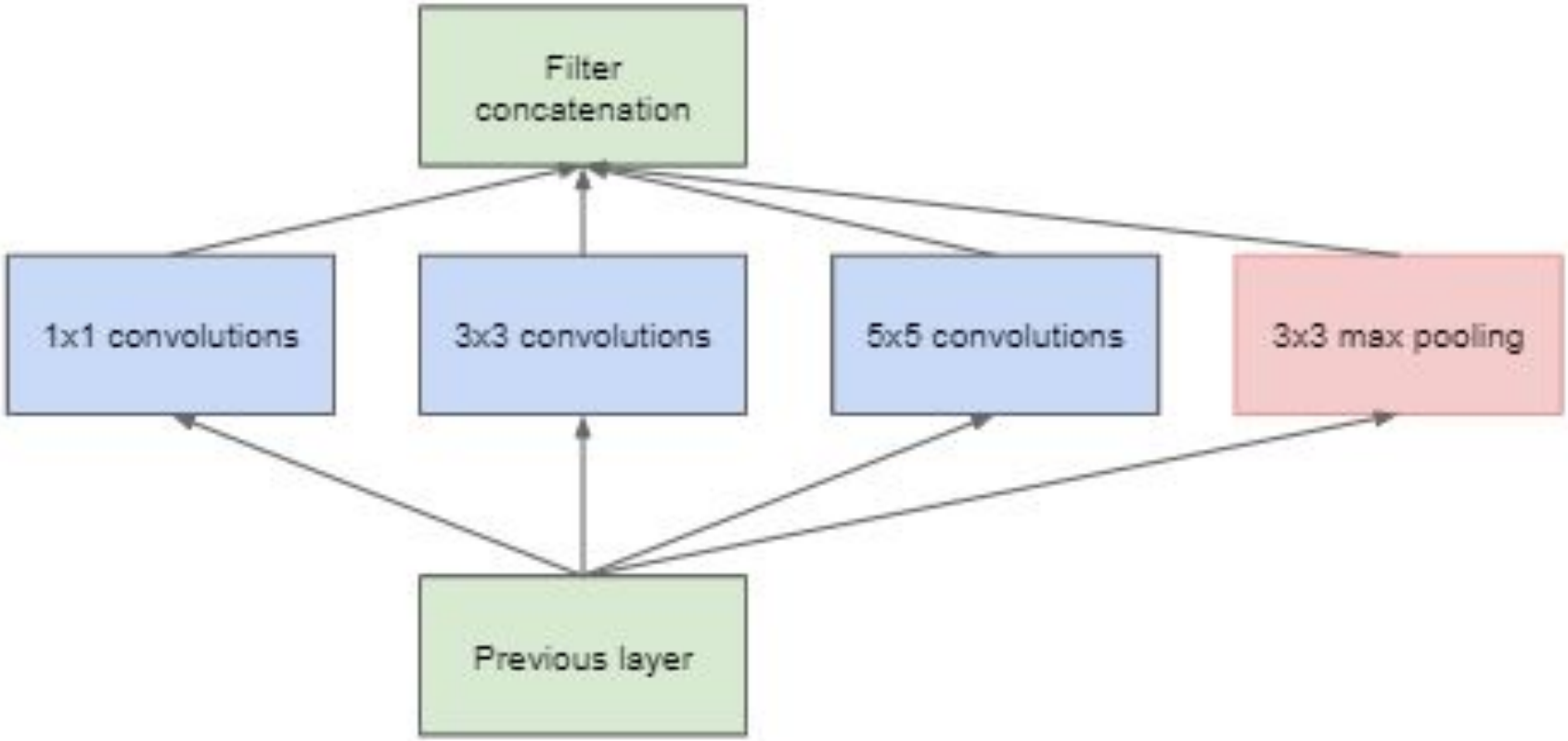
- Zero additional trainable parameters
- Reduces overfitting
- Improves top-1 accuracy by approximately 0.6%.
  - Example: ImageNet validation set ≈ 50,000 images
  - $0.6\% \times 50000 = 300$
  - So about 300 additional images are classified correctly.

**3. Inception Module:** The Inception module is the architectural core of GoogLeNet.

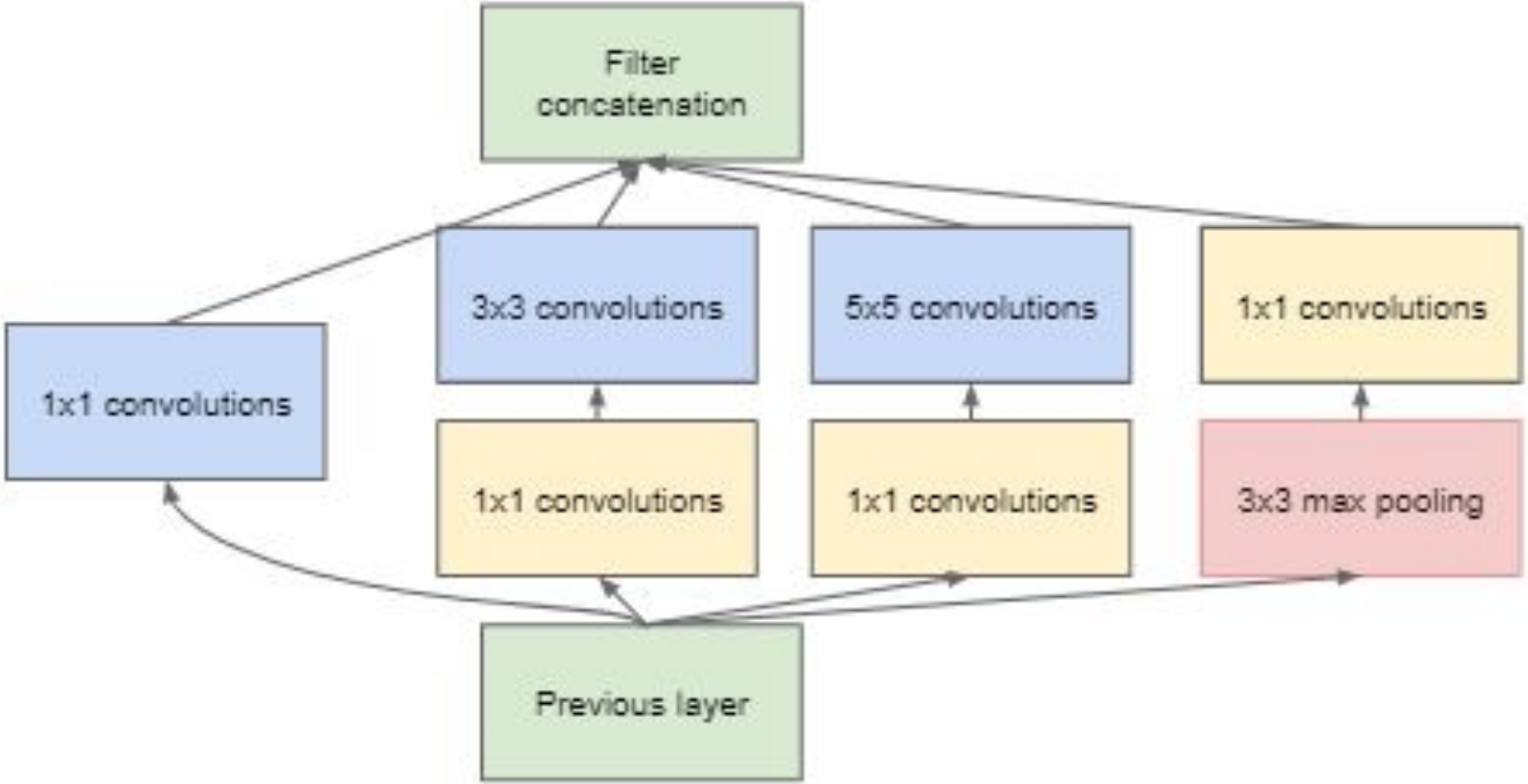
- It processes the input using multiple types of operations in parallel, including 1×1, 3×3, 5×5 convolutions and 3×3 max pooling.
- The outputs from all paths are concatenated depth-wise.

**Purpose:** Capture features at multiple scales effectively.

**Advantage:** Improves representational power without dramatically increasing computation.




(a) Inception module, naïve version




(b) Inception module with dimension reductions

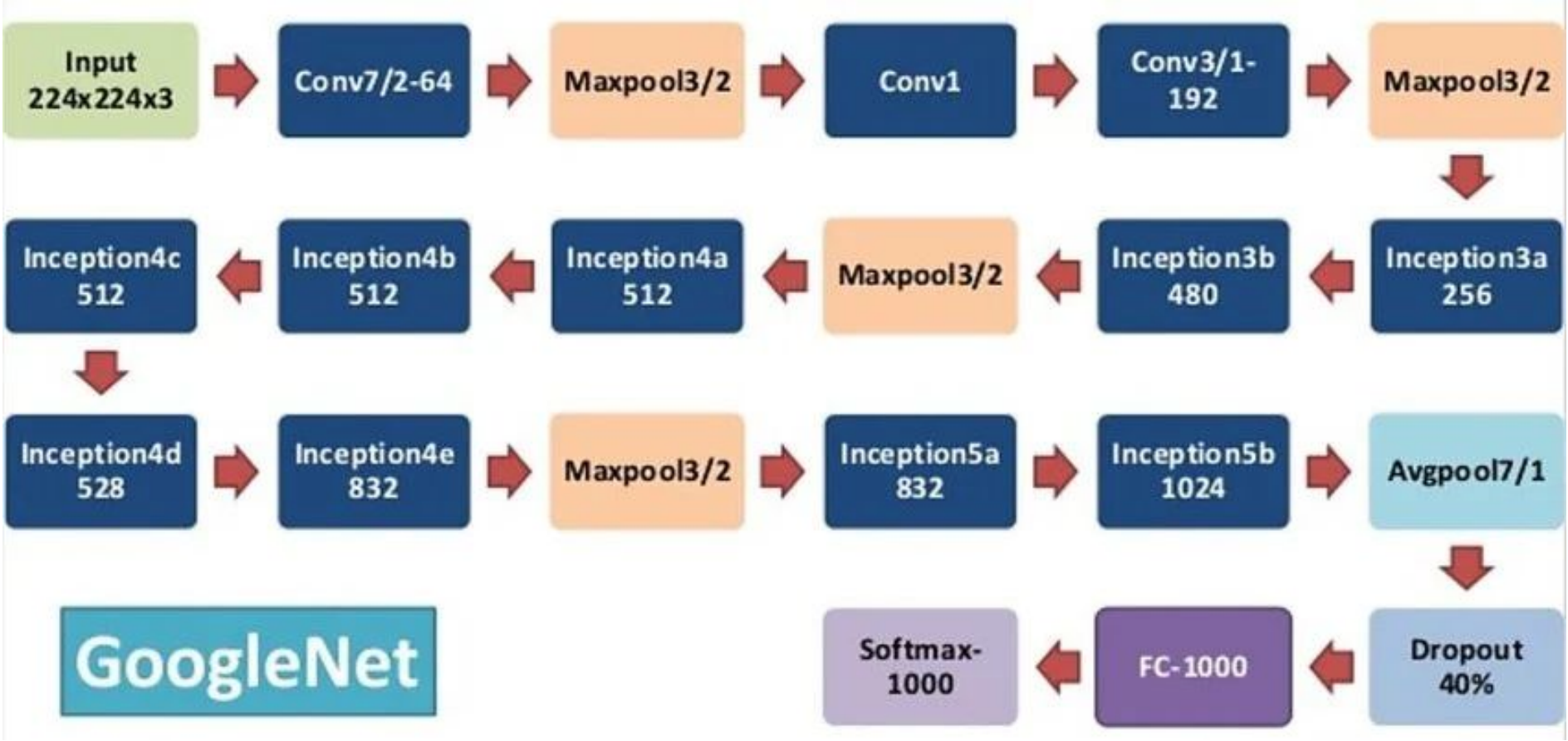
**4. Auxiliary Classifiers:** To address the vanishing gradient problem during training. Smaller classifiers are introduced in between that are active only during training and help regularize the network.

- Structure of Each Auxiliary Classifier:
  - Average pooling layer (5×5, stride 3)
  - 1×1 convolution (128 filters, ReLU)
  - Fully connected layer (1024 units, ReLU)
  - Dropout layer (dropout rate = 0.7)
  - Fully connected softmax layer (1000 classes)
  - The auxiliary losses are added to the main loss with a weight of 0.3 to stabilize training.

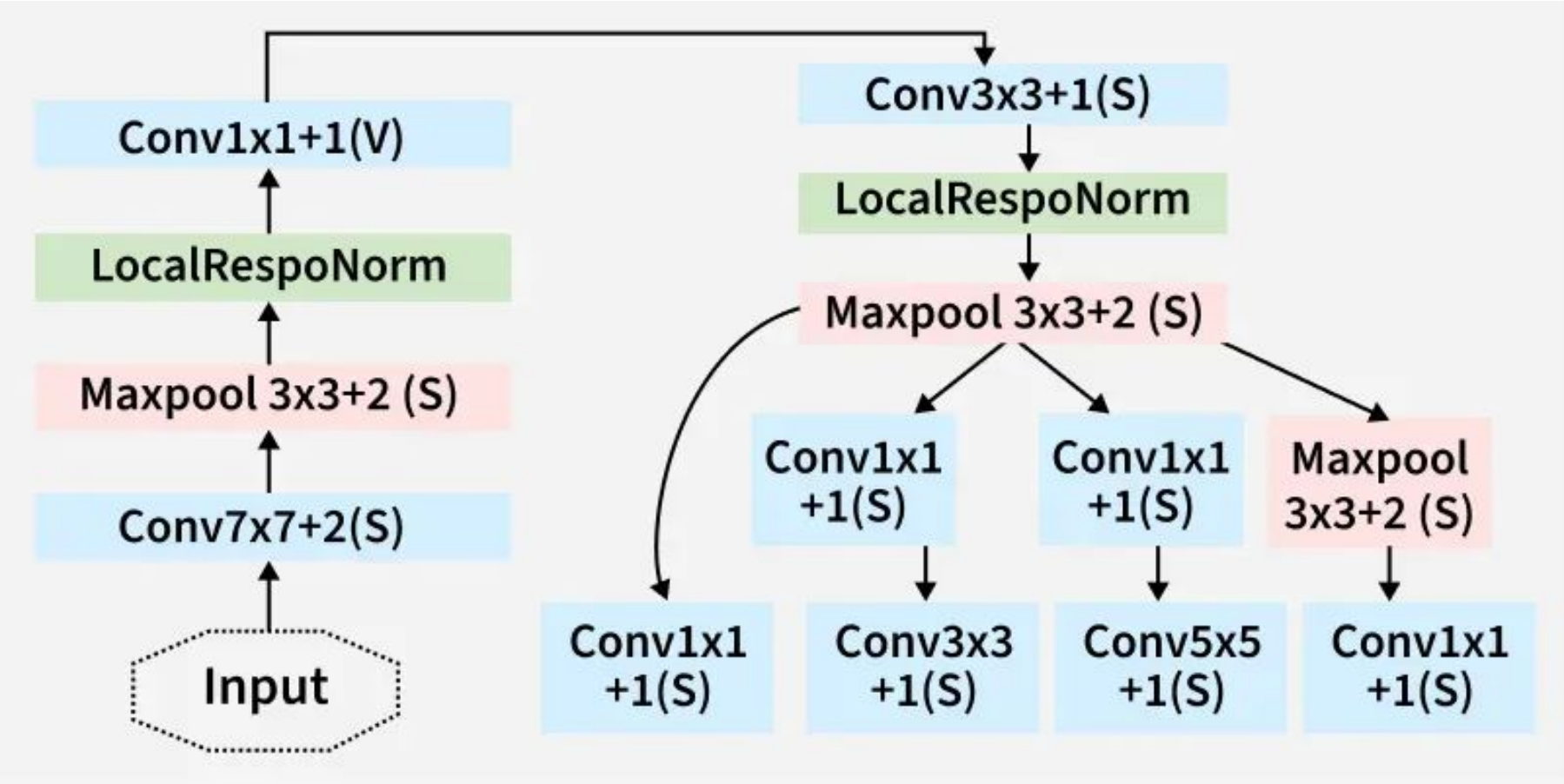
# 5. Model Architecture

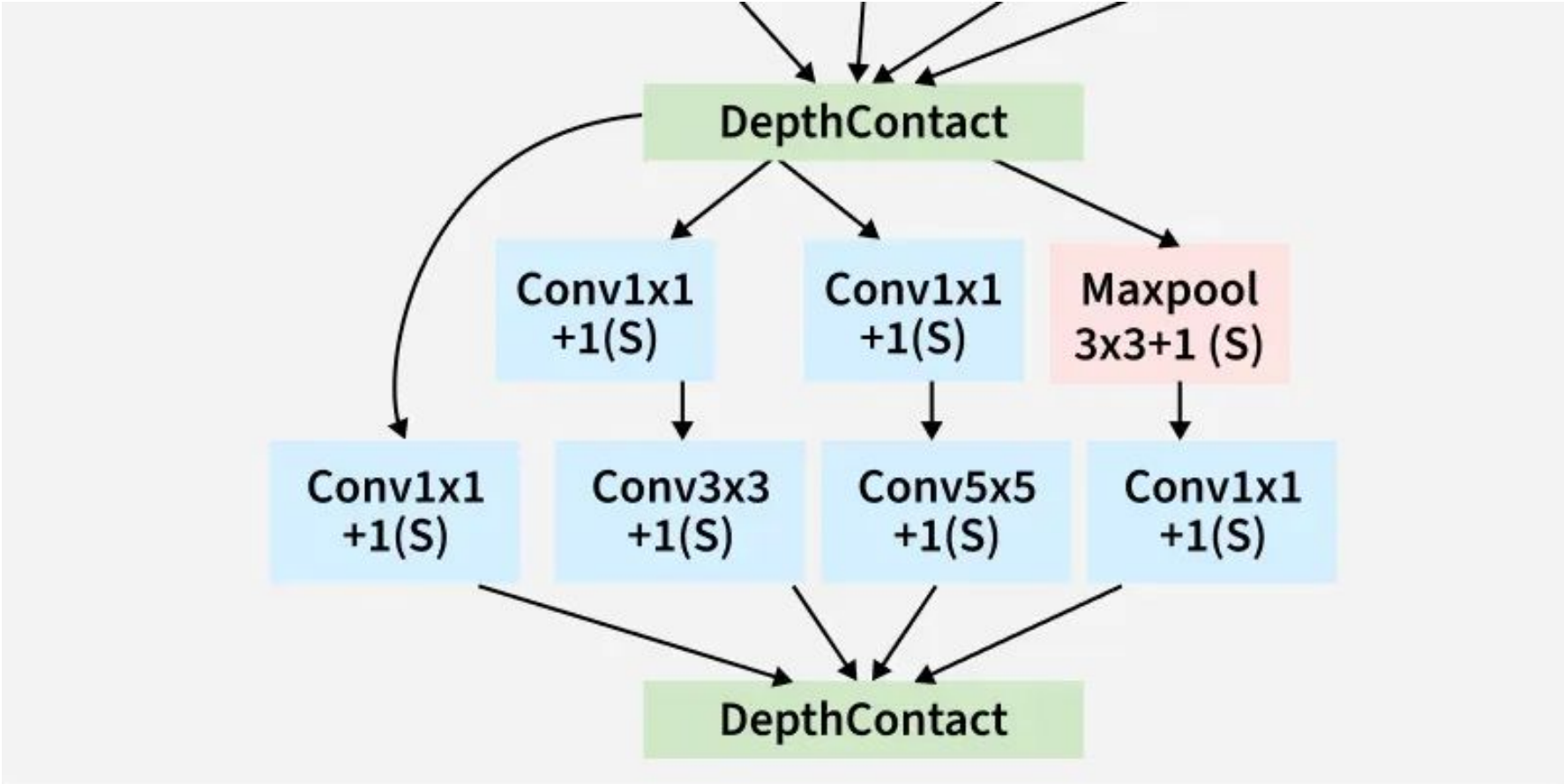
- GoogLeNet is a 22-layer deep network (excluding pooling layers) that emphasizes computational efficiency, making it feasible to run even on hardware with limited resources.
- The architecture also contains two auxiliary classifier layer connected to the output of Inception (4a) and Inception (4d) layers.

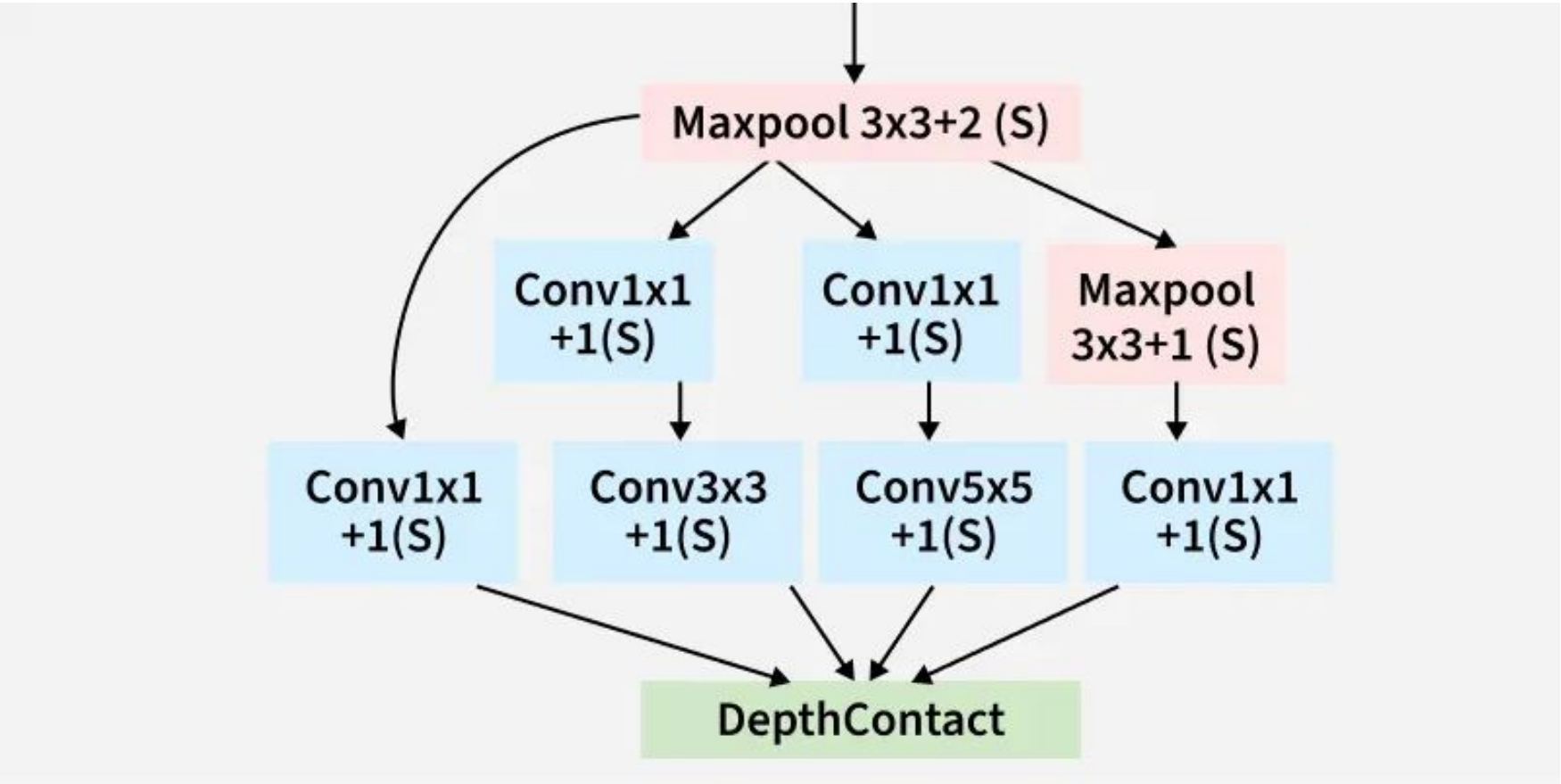
| type | patch size/ stride | output size | depth | #1×1 | #3×3 reduce | #3×3 | #5×5 reduce | #5×5 | pool proj | params | ops |
|---|---|---|---|---|---|---|---|---|---|---|---|
| convolution | 7×7/2 | 112×112×64 | 1 | | | | | | | 2.7K | 34M |
| max pool | 3×3/2 | 56×56×64 | 0 | | | | | | | | |
| convolution | 3×3/1 | 56×56×192 | 2 | | 64 | 192 | | | | 112K | 360M |
| max pool | 3×3/2 | 28×28×192 | 0 | | | | | | | | |
| inception (3a) | | 28×28×256 | 2 | 64 | 96 | 128 | 16 | 32 | 32 | 159K | 128M |
| inception (3b) | | 28×28×480 | 2 | 128 | 128 | 192 | 32 | 96 | 64 | 380K | 304M |
| max pool | 3×3/2 | 14×14×480 | 0 | | | | | | | | |
| inception (4a) | | 14×14×512 | 2 | 192 | 96 | 208 | 16 | 48 | 64 | 364K | 73M |
| inception (4b) | | 14×14×512 | 2 | 160 | 112 | 224 | 24 | 64 | 64 | 437K | 88M |
| inception (4c) | | 14×14×512 | 2 | 128 | 128 | 256 | 24 | 64 | 64 | 463K | 100M |
| inception (4d) | | 14×14×528 | 2 | 112 | 144 | 288 | 32 | 64 | 64 | 580K | 119M |
| inception (4e) | | 14×14×832 | 2 | 256 | 160 | 320 | 32 | 128 | 128 | 840K | 170M |
| max pool | 3×3/2 | 7×7×832 | 0 | | | | | | | | |
| inception (5a) | | 7×7×832 | 2 | 256 | 160 | 320 | 32 | 128 | 128 | 1072K | 54M |
| inception (5b) | | 7×7×1024 | 2 | 384 | 192 | 384 | 48 | 128 | 128 | 1388K | 71M |
| avg pool | 7×7/1 | 1×1×1024 | 0 | | | | | | | | |
| dropout (40%) | | 1×1×1024 | 0 | | | | | | | | |
| linear | | 1×1×1000 | 1 | | | | | | | 1000K | 1M |
| softmax | | 1×1×1000 | 0 | | | | | | | | |

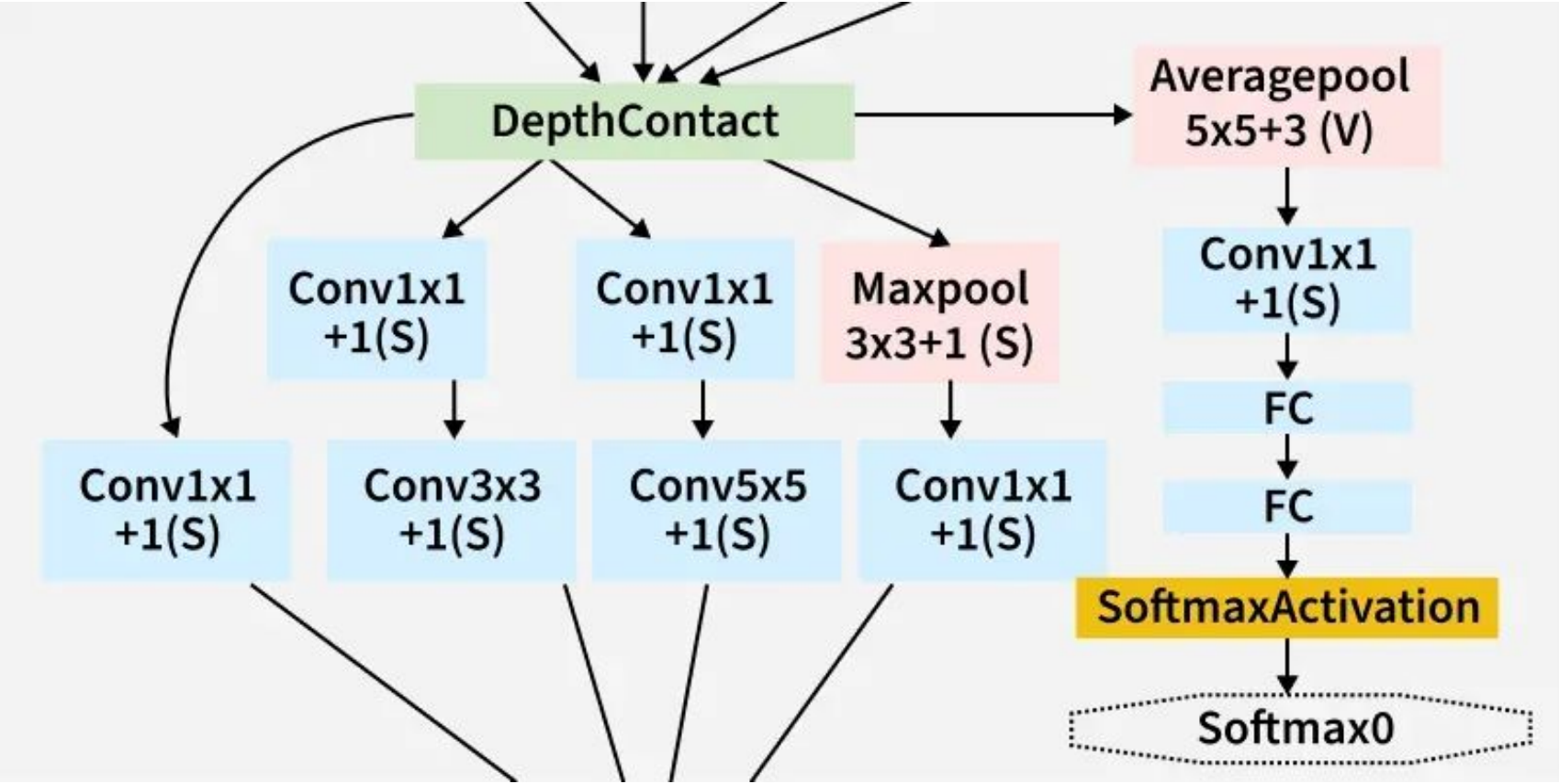

Input 224x224x3
Conv7/2-64
Maxpool3/2
Conv1
Conv3/1-192
Maxpool3/2
Inception3a 256
Inception3b 480
Maxpool3/2
Inception4a 512
Inception4b 512
Inception4c 512
Inception4d 528
Inception4e 832
Maxpool3/2
Inception5a 832
Inception5b 1024
Avgpool7/1
Dropout 40%
FC-1000
Softmax-1000
GoogleNet

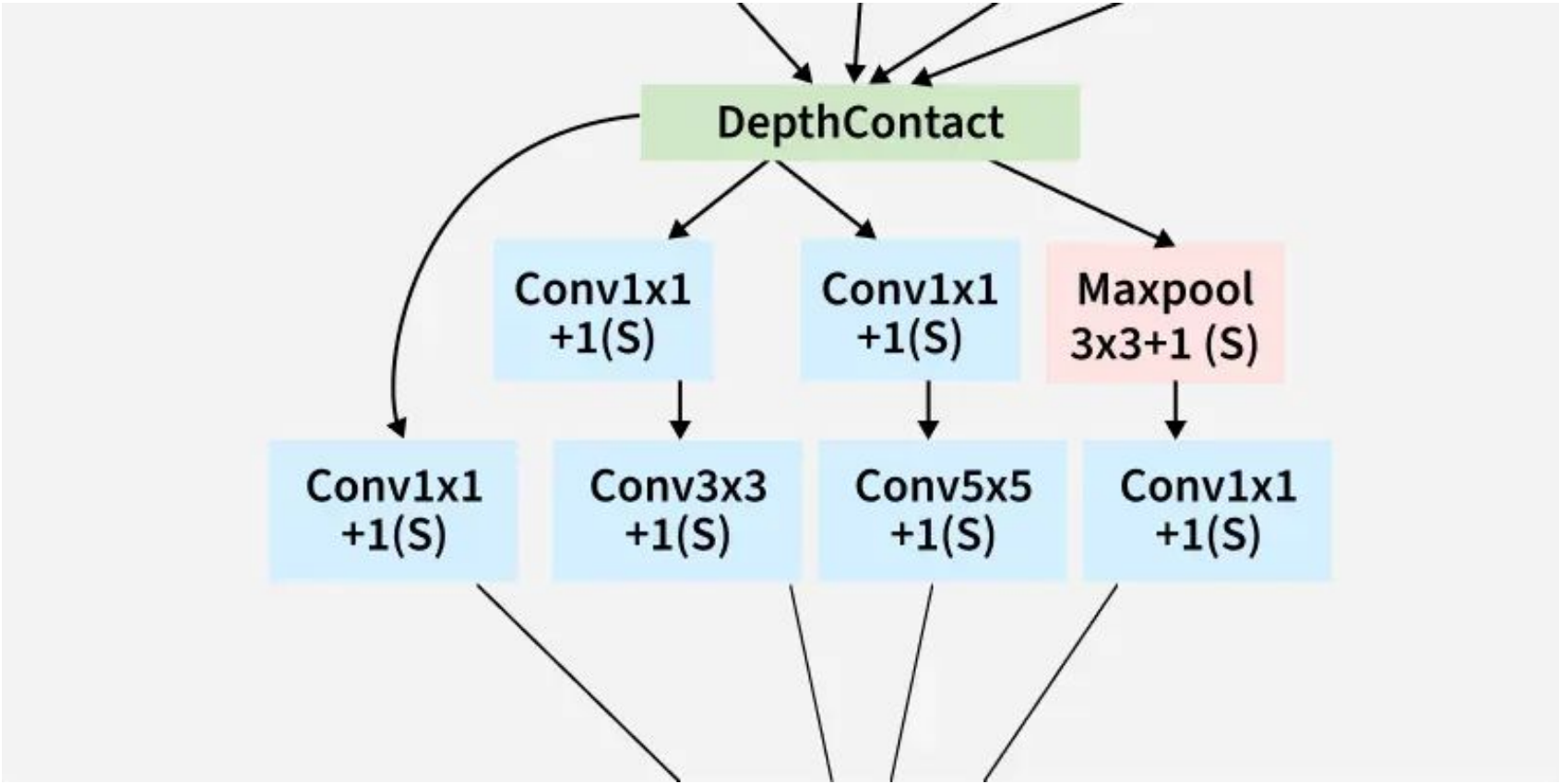
# Inception V1 architecture

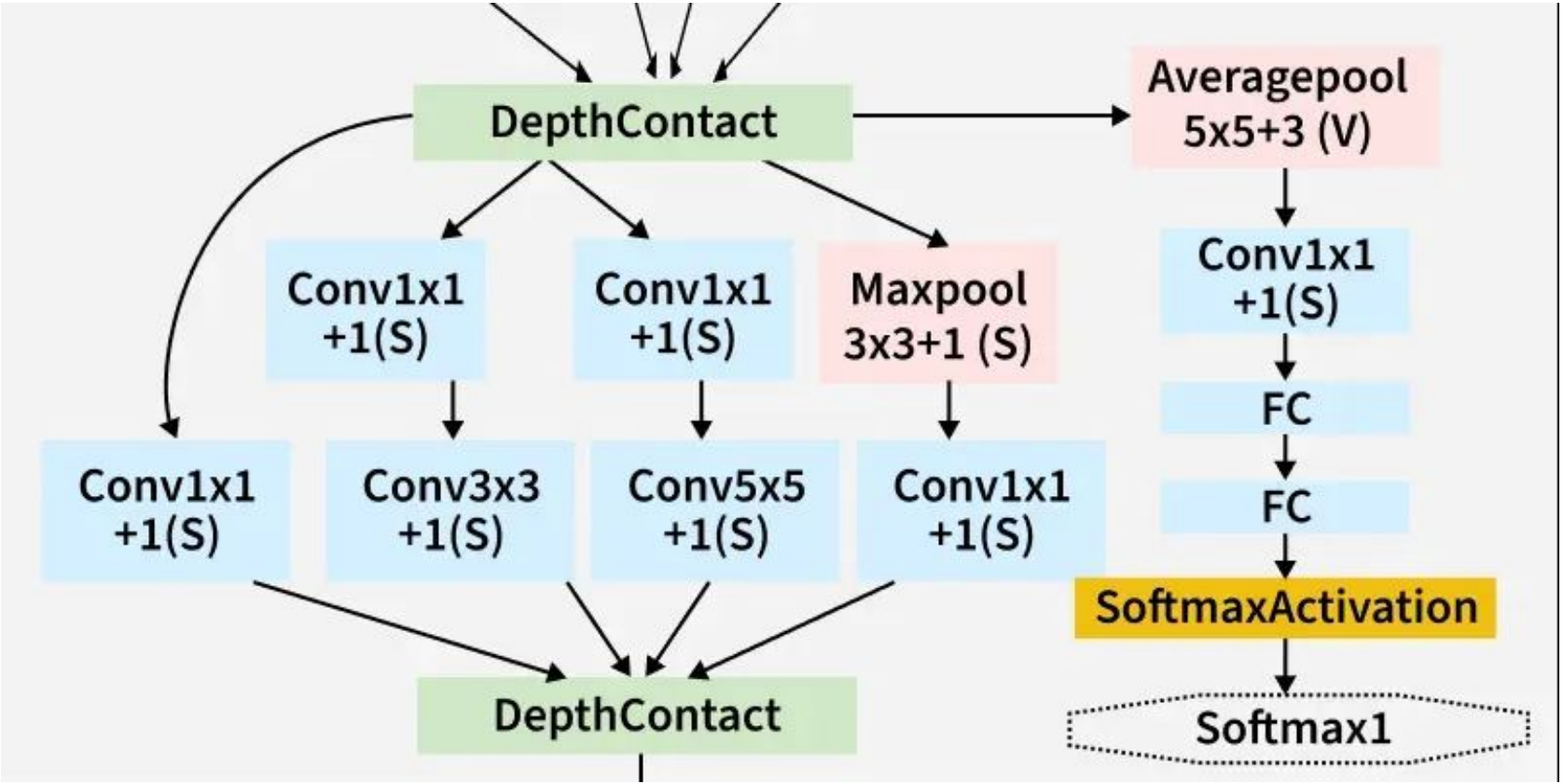


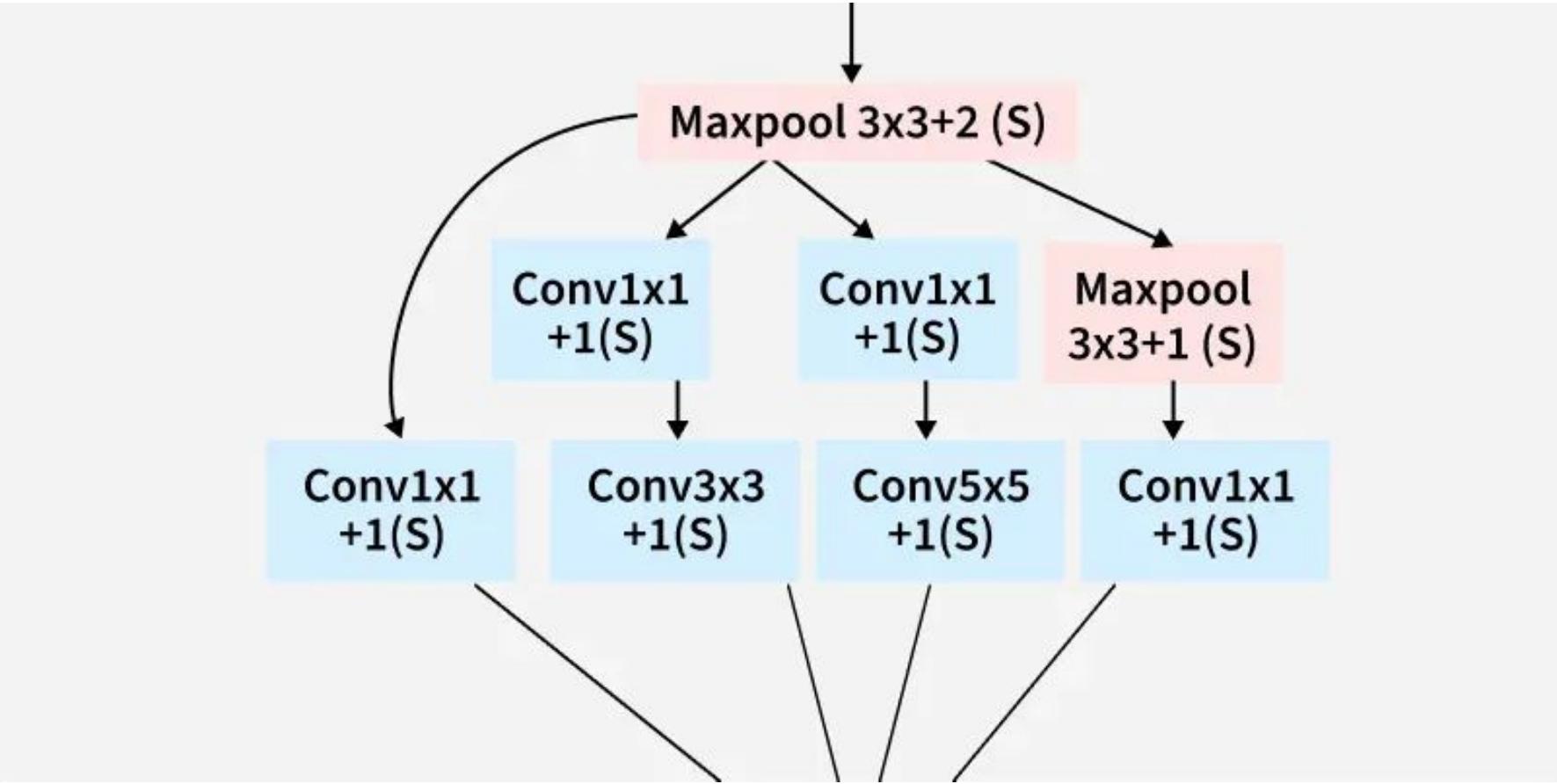

DepthContact
Conv1x1
+1(S)
Conv1x1
+1(S)
Maxpool
3x3+1 (S)
Conv1x1
+1(S)
Conv3x3
+1(S)
Conv5x5
+1(S)
Conv1x1
+1(S)
DepthContact

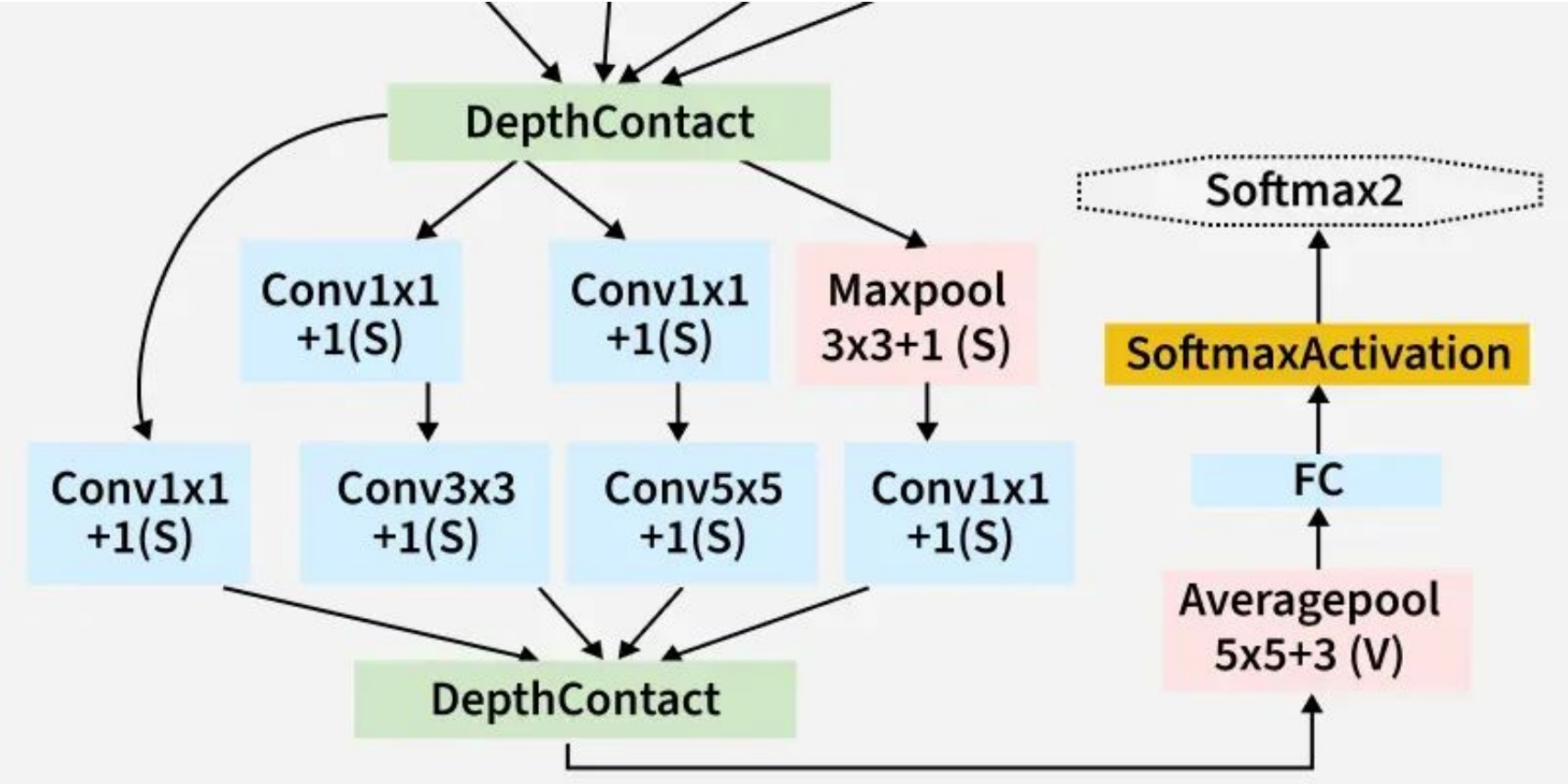

Maxpool 3x3+2 (S)
Conv1x1 +1(S)
Conv1x1 +1(S)
Maxpool 3x3+1 (S)
Conv1x1 +1(S)
Conv3x3 +1(S)
Conv5x5 +1(S)
Conv1x1 +1(S)
DepthContact


DepthContact
Averagepool
5x5+3 (V)
Conv1x1
+1(S)
Conv1x1
+1(S)
Maxpool
3x3+1 (S)
Conv1x1
+1(S)
FC
FC
Conv1x1
+1(S)
Conv3x3
+1(S)
Conv5x5
+1(S)
Conv1x1
+1(S)
SoftmaxActivation
Softmax0


DepthContact
Conv1x1
+1(S)
Conv1x1
+1(S)
Maxpool
3x3+1 (S)
Conv1x1
+1(S)
Conv3x3
+1(S)
Conv5x5
+1(S)
Conv1x1
+1(S)


DepthContact
Averagepool
5x5+3 (V)
Conv1x1
+1(S)
Conv1x1
+1(S)
Maxpool
3x3+1 (S)
Conv1x1
+1(S)
FC
FC
Conv1x1
+1(S)
Conv3x3
+1(S)
Conv5x5
+1(S)
Conv1x1
+1(S)
SoftmaxActivation
DepthContact
Softmax1


Maxpool 3x3+2 (S)
Conv1x1 +1(S)
Conv1x1 +1(S)
Maxpool 3x3+1 (S)
Conv1x1 +1(S)
Conv3x3 +1(S)
Conv5x5 +1(S)
Conv1x1 +1(S)


DepthContact
Conv1x1
+1(S)
Conv1x1
+1(S)
Maxpool
3x3+1 (S)
Conv1x1
+1(S)
Conv3x3
+1(S)
Conv5x5
+1(S)
Conv1x1
+1(S)
DepthContact
Softmax2
SoftmaxActivation
FC
Averagepool
5x5+3 (V)

# Key highlights of the architecture:

- **Input Layer:** Accepts a 224×224 RGB image as input.
- **Initial Convolutions and Pooling:** Applies a series of standard convolutional and max pooling layers to downsample the input and extract low-level features.
- **Local Response Normalization (LRN):** Normalizes the feature maps early in the network to improve generalization.

- **Inception Modules:** Each module processes the input through 1×1, 3×3, and 5×5 convolutions, as well as 3×3 max pooling, all in parallel. The outputs are concatenated along the depth dimension, allowing the network to capture both fine and coarse features.
- **Auxiliary Classifiers:** Appear as smaller branches connected to intermediate layers of the network. Include average pooling, 1×1 convolutions, fully connected layers, and softmax outputs.

- **Final Layers:** Uses global average pooling (7×7) to reduce each feature map to a single value. Followed by a fully connected layer and a softmax activation to produce the final classification output.

# Performance and Results

- Winner of ILSVRC 2014 in both classification and detection tasks
- Achieved a top-5 error rate of 6.67% in image classification
- An ensemble of six GoogLeNet models achieved 43.9% mAP (mean Average Precision) on the ImageNet detection task

# GoogLeNet Classification top-5 Error

| Team | Year | Place | Error (top-5) | Uses external data |
|---|---|---|---|---|
| SuperVision | 2012 | 1st | 16.4% | no |
| SuperVision | 2012 | 1st | 15.3% | Imagenet 22k |
| Clarifai | 2013 | 1st | 11.7% | no |
| Clarifai | 2013 | 1st | 11.2% | Imagenet 22k |
| MSRA | 2014 | 3rd | 7.35% | no |
| VGG | 2014 | 2nd | 7.32% | no |
| GoogLeNet | 2014 | 1st | 6.67% | no |

# GoogLeNet Detection Performance

| Team | Year | Place | mAP | external data | ensemble | approach |
|---|---|---|---|---|---|---|
| UvA-Euvision | 2013 | 1st | 22.6% | none | ? | Fisher vectors |
| Deep Insight | 2014 | 3rd | 40.5% | ImageNet 1k | 3 | CNN |
| CUHK DeepID-Net | 2014 | 2nd | 40.7% | ImageNet 1k | ? | CNN |
| GoogLeNet | 2014 | 1st | 43.9% | ImageNet 1k | 6 | CNN |

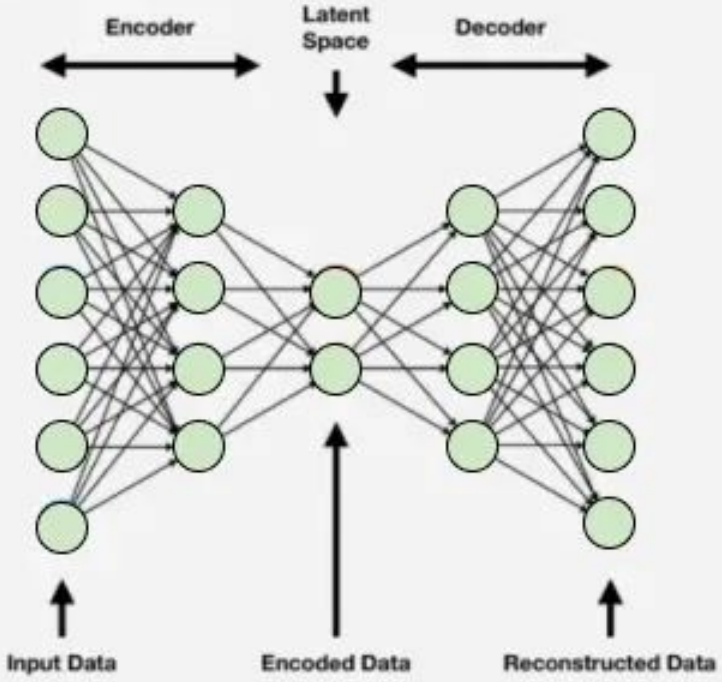
# Autoencoders

- Autoencoders are a special type of neural networks that learn to compress data into a compact form and then reconstruct it to closely match the original input.
- They consist of an:
  - Encoder that captures important features by reducing dimensionality.
  - Decoder that rebuilds the data from this compressed representation.

- The model trains by minimizing reconstruction error using loss functions like Mean Squared Error or Binary Cross-Entropy.
- These are applied in tasks such as noise removal, error detection and feature extraction where capturing efficient data representations is important

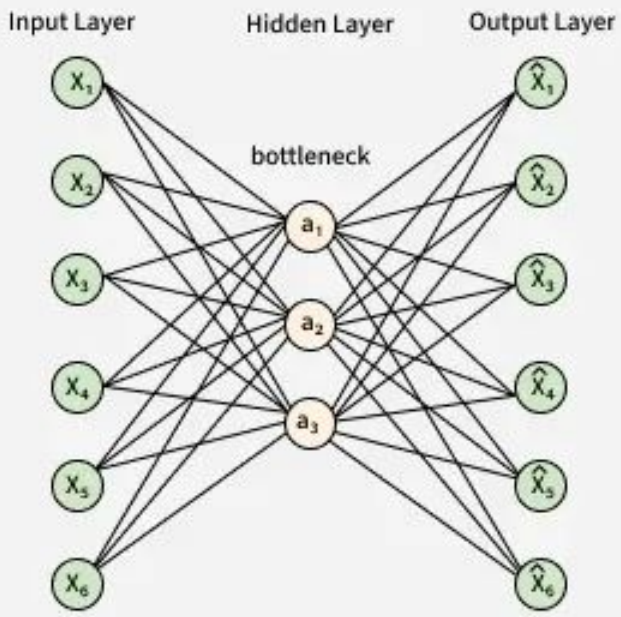
# What is an Autoencoder?

- Learns to compress and reconstruct data
- Two parts: Encoder (compress), Decoder (reconstruct)
- Used for noise removal, anomaly detection & feature extraction



# How Does it Work?

- Encoder compresses input into smaller features
- Bottleneck holds compact data representation
- Decoder rebuilds original data from compressed form



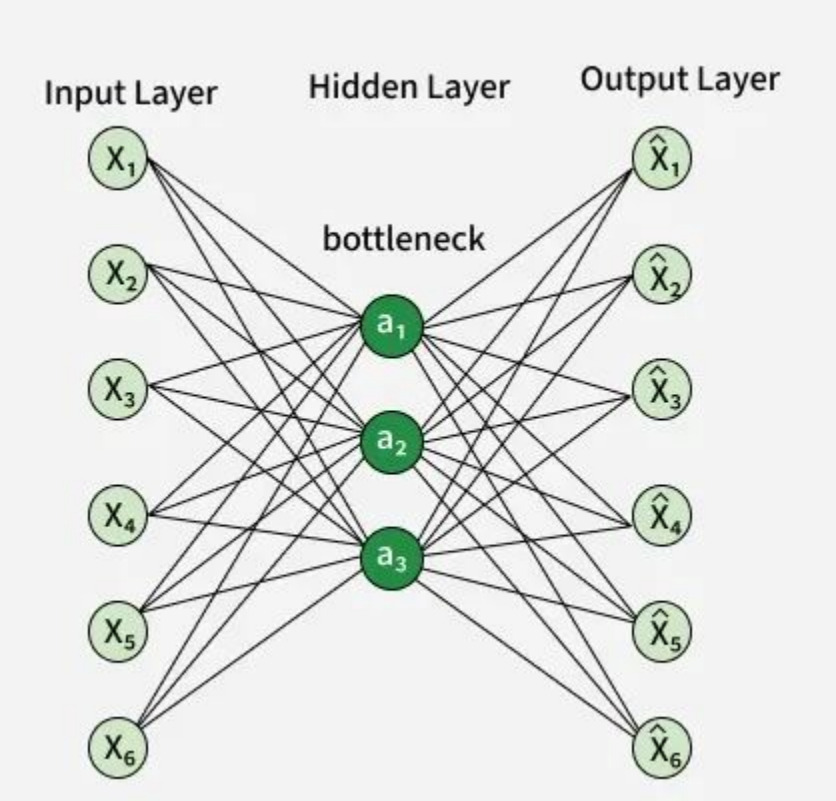
# Benefits and Challenges of Autoencoder

- Captures important features efficiently
- Useful for reducing noise
- Useful for detecting anomalies

- Needs lots of data
- Can blur outputs (imperfect reconstruction)
- May memorize input (overfitting risk)

# Architecture of Autoencoder

An autoencoder's architecture consists of three main components that work together to compress and then reconstruct data



# 1. Encoder

- It compress the input data into a smaller, more manageable form by reducing its dimensionality while preserving important information.
- It has three layers which are:
  - *Input Layer:* This is where the original data enters the network. It can be images, text features or any other structured data.

- *Hidden Layers:* These layers perform a series of transformations on the input data. Each hidden layer applies weights and activation functions to capture important patterns, progressively reducing the data's size and complexity.
- *Output(Latent Space):* The encoder outputs a compressed vector known as the latent representation or encoding. This vector captures the important features of the input data in a condensed form helps in filtering out noise and redundancies.

# 2. Bottleneck (Latent Space)

- It is the smallest layer of the network which represents the most compressed version of the input data.
- It serves as the information bottleneck which force the network to prioritize the most significant features.
- This compact representation helps the model learn the underlying structure and key patterns of the input helps in enabling better generalization and efficient data encoding.

# 3. Decoder

- It is responsible for taking the compressed representation from the latent space and reconstructing it back into the original data form.
- *Hidden Layers:* These layers progressively expand the latent vector back into a higher-dimensional space. Through successive transformations decoder attempts to restore the original data shape and details

- *Output Layer:* The final layer produces the reconstructed output which aims to closely resemble the original input. The quality of reconstruction depends on how well the encoder-decoder pair can minimize the difference between the input and output during training.

# Loss Function in Autoencoder Training

- During training an autoencoder's goal is to minimize the reconstruction loss which measures how different the reconstructed output is from the original input.
- The choice of loss function depends on the type of data being processed:
  - *Mean Squared Error (MSE):* This is commonly used for continuous data. It measures the average squared differences between the input and the reconstructed data.

- *Binary Cross-Entropy:* Used for binary data (0 or 1 values). It calculates the difference in probability between the original and reconstructed output.

- During training the network updates its weights using backpropagation to minimize this reconstruction loss. By doing this it learns to extract and retain the most important features of the input data which are encoded in the latent space.

# Efficient Representations in Autoencoders

- Constraining an autoencoder helps it learn meaningful and compact features from the input data which leads to more efficient representations. After training, only the encoder part is used to encode similar data for future tasks.
- Various techniques are used to achieve this are as follows:
  - *Keep Small Hidden Layers:* Limiting the size of each hidden layer forces the network to focus on the most important features. Smaller layers reduce redundancy and allows efficient encoding.

- *Regularization:* Techniques like L1 or L2 regularization add penalty terms to the loss function. This prevents overfitting by removing excessively large weights which helps in ensuring the model to learns general and useful representations.
- *Denoising:* In denoising autoencoders random noise is added to the input during training. It learns to remove this noise during reconstruction which helps it focus on core, noise-free features and helps in improving robustness.

- *Tuning the Activation Functions:* Adjusting activation functions can promote sparsity by activating only a few neurons at a time. This sparsity reduces model complexity and forces the network to capture only the most relevant features.

# Types of Autoencoders

1. **Denoising Autoencoder**
   Denoising Autoencoder is trained to handle corrupted or noisy inputs, it learns to remove noise and helps in reconstructing clean data. It prevent the network from simply memorizing the input and encourages learning the core features.
2. **Sparse Autoencoder**
   Sparse Autoencoder contains more hidden units than input features but only allows a few neurons to be active simultaneously. This sparsity is controlled by zeroing some hidden units, adjusting activation functions or adding a sparsity penalty to the loss function.

3. **Variational Autoencoder**

Variational autoencoder (VAE) makes assumptions about the probability distribution of the data and tries to learn a better approximation of it. It uses stochastic gradient descent to optimize and learn the distribution of latent variables. They are used for generating new data such as creating realistic images or text.

It assumes that the data is generated by a Directed Graphical Model and tries to learn an approximation to $q_\phi(z|x)$ to the conditional property $q_\theta(z|x)$ where $\phi$ and $\theta$ are the parameters of the encoder and the decoder respectively.

4. **Convolutional Autoencoder**

   Convolutional autoencoder uses convolutional neural networks (CNNs) which are designed for processing images. The encoder extracts features using convolutional layers and the decoder reconstructs the image through deconvolution also called as upsampling.

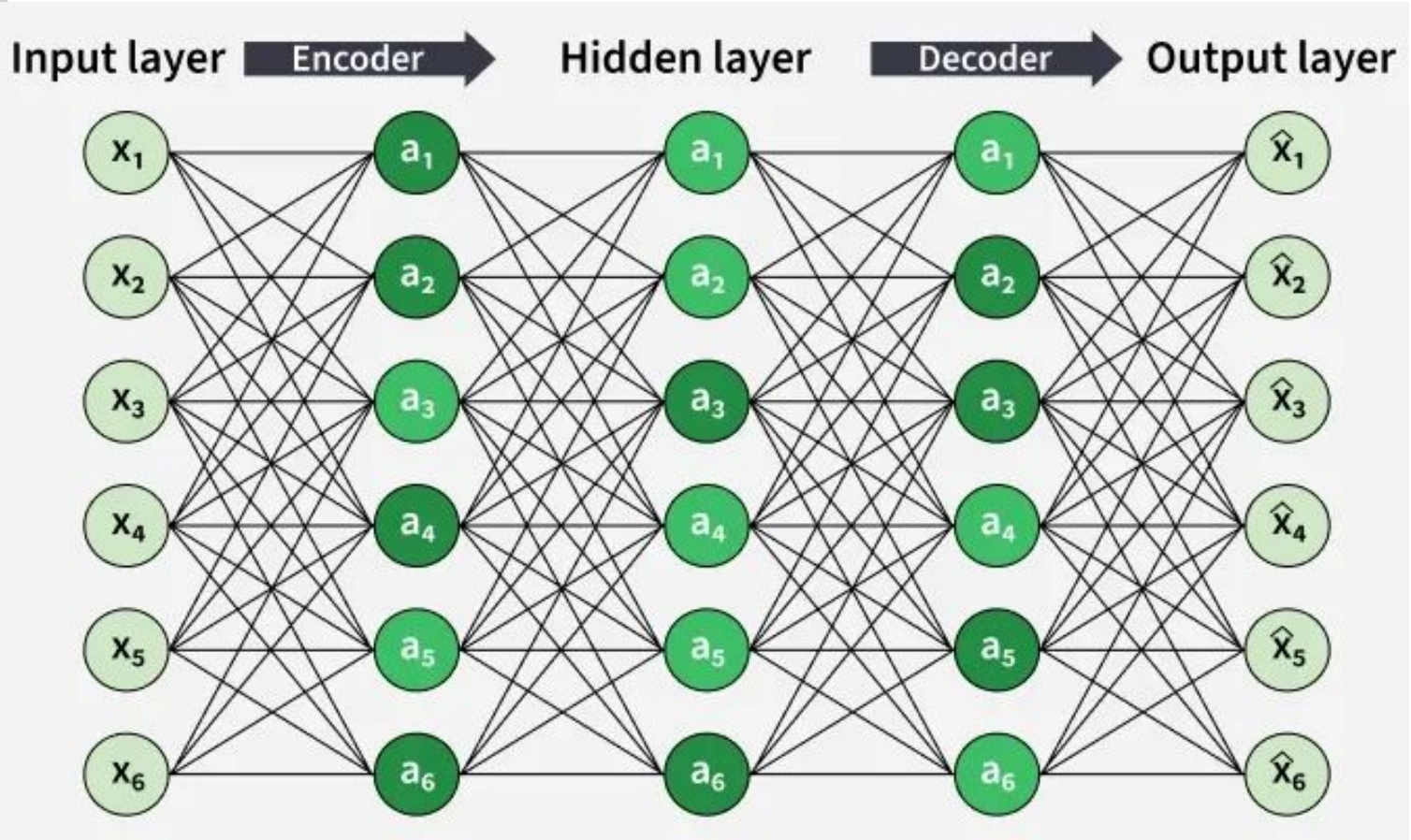
# Limitations

- **Memorizing Instead of Learning Patterns:** sometimes memorizes the training data rather than learning meaningful patterns reducing the ability to generalize new data.
- **Reconstructed Data Might Not Be Perfect:** Output may be blurry or distorted with noisy inputs or if the model architecture lacks sufficient complexity to capture all details.
- **Requires a Large Dataset and Good Parameter Tuning:** It require large amounts of data and careful parameter tuning (latent dimension size, learning rate, etc) to perform well. Insufficient data or poor tuning can result in weak feature representations.

# Sparse Autoencoders

- Sparse autoencoder introduces a sparsity constraint on the hidden layer, forcing only a few neurons to activate at a time.
- Restricts hidden layer activation so the network focuses only on the most informative features.
- Prevents the autoencoder from overfitting or copying inputs directly.
- Helps extract meaningful structure from high-dimensional data.
- The sparsity is typically controlled through regularization techniques like L1 penalty or KL divergence.

# Hidden layer learns a compressed internal representation by keeping most hidden units inactive (sparse).



# Importance of Sparse Autoencoders

- Sparse autoencoders are useful for learning compact and meaningful representations of data without supervision.
- **Feature Learning:** Extracts important features from unlabeled data for tasks like classification and clustering.
- **Dimensionality Reduction:** Compresses high dimensional data while preserving essential information.
- **Anomaly Detection:** Identifies unusual inputs that deviate from learned patterns.

- **Denoising:** Reconstructs clean data from noisy inputs.
- **Pretraining Networks:** Provides a strong weight initialization for deep networks.
- **Interpretable Representations:** Sparse activations highlight the most informative features.

# How it Works

- Sparse autoencoders combine an encoder, decoder and sparsity constrained loss function to learn compact meaningful representations.
- The loss function used is: $L = \|X - \hat{X}\|^2 + \lambda \cdot \text{Penalty}(s)$
  - Where
    - X: Input data
    - $\hat{X}$: Reconstructed output
    - λ: Regularization parameter
    - Penalty(s): A function penalizing deviations from sparsity

**Working:**

- Encoder compresses the input into a low dimensional latent representation.
- Decoder reconstructs the input from the encoded features.
- Sparsity Constraint ensures only a small number of hidden neurons activate for each input.

# Methods to Enforce Sparsity

- Sparsity is enforced to ensure the autoencoder learns high level features by activating only a small subset of neurons
- **L1 Regularization:** Applies a penalty proportional to the absolute activations or weights, encouraging the network to use fewer neurons.
- **KL Divergence:** Measures how far the average activation deviates from the target sparsity level, ensuring that only a subset of neurons activates at a time.
- **Sparsity Proportion:** Defines the desired activation frequency of hidden units across training samples.

# How Sparse Autoencoders Are Trained

- Training a sparse autoencoder follows the same workflow as a standard autoencoder but includes an additional step to enforce sparsity on the hidden layer:

1. **Initialization:** The network weights are initialized randomly or using pre-trained models to provide a stable starting point.
2. **Forward Pass:** Input data is passed through the encoder to obtain the latent (compressed) representation and then through the decoder to reconstruct the original input.

3. **Loss Calculation:** The total loss combines:
   *Reconstruction error:* MSE between X and $\hat{X}$
   *Sparsity penalty:* Uses KL Divergence or L1 regularization to keep most hidden units inactive.
4. **Backpropagation and Optimization:** Gradients are computed from the combined loss and used to update the network weights, ensuring the model learns both accurate reconstruction and sparse feature representations.

# Applications

- **Feature Extraction:** Learns important features from raw data for tasks like classification and clustering.
- **Dimensionality Reduction:** Reduces high dimensional data while preserving key information.
- **Anomaly Detection:** Detects unusual patterns by identifying inputs that cannot be well reconstructed.
- **Image Denoising:** Removes noise from images by learning the underlying clean representation.
- **Pretraining for Deep Networks:** Initializes weights in deep networks to improve convergence and performance.

# Advantages

- **Efficient Feature Learning:** Learns compact and meaningful representations of data.
- **Reduces Overfitting:** Sparsity constraint prevents the network from memorizing data.
- **Handles High Dimensional Data:** Can compress large input data effectively.
- **Improves Downstream Tasks:** Enhances performance in classification, clustering and anomaly detection.
- **Flexible Architecture:** Can be combined with other models or used for pretraining deep networks.

# Challenges

- **Choosing Sparsity Level:** Setting an appropriate sparsity target can be tricky and impacts performance.
- **Computational Complexity:** Training can be slow for large datasets or deep architectures.
- **Risk of Underfitting:** Too much sparsity may cause the model to miss important features.
- **Hyperparameter Tuning:** Requires careful tuning of learning rate, hidden units and regularization.
- **Sensitive to Initialization:** Poor weight initialization can lead to vanishing/exploding gradients.

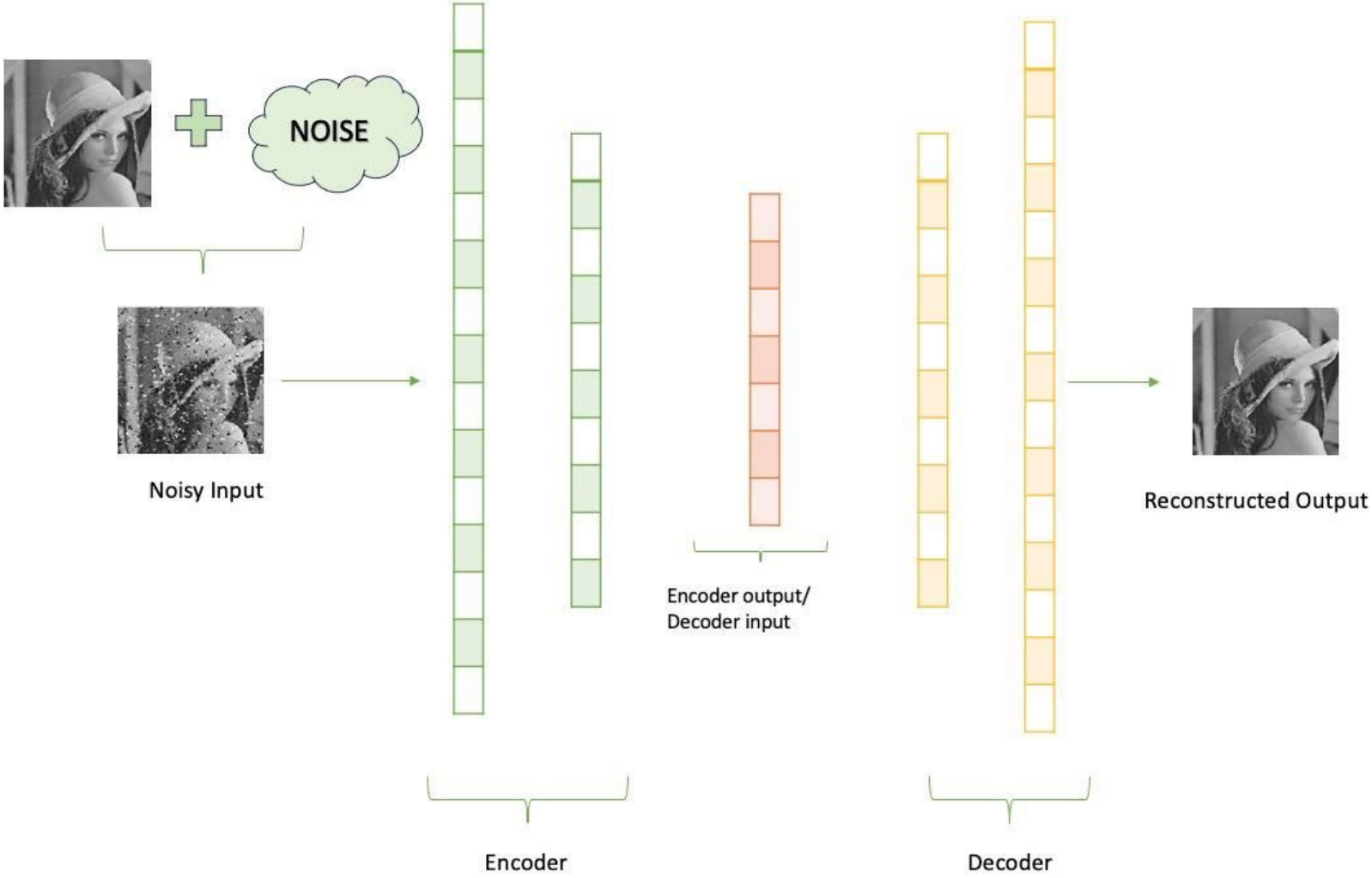
# Denoising Autoencoders (DAE)

- Autoencoders are neural networks for unsupervised learning that encode input data into a low-dimensional representation and then decode it to reconstruct the original input by minimizing reconstruction error.
- If the hidden layer is too large, autoencoders may simply learn to replicate the input perfectly, functioning as an identity mapping and failing to extract meaningful features.

- Denoising autoencoders address this by providing a deliberately noisy or corrupted version of the input to the encoder, but still using the original, clean input for calculating loss.
- This trains the model to learn useful, robust features and reduces the chance of simply replicating the input.

# Architecture of DAE

- Consists of two main components:
  - **Encoder**
    - A neural network (one or more layers) that transforms noisy input data into a lower-dimensional encoding.
    - Noise can be introduced by adding Gaussian noise or randomly masking/missing some inputs.
  - **Decoder**
    - A neural network (one or more layers) that reconstructs the original data from the encoding.
    - The loss is calculated between the decoder output and the original clean input, not the noisy one.


NOISE
Noisy Input
Encoder output/
Decoder input
Reconstructed Output
Encoder
Decoder

# Applications

- **Image Denoising:** Removing noise from images to restore clear, high-quality visuals.
- **Data Imputation:** Filling in missing values or reconstructing incomplete data entries.
- **Feature Extraction:** Learning robust features that improve performance for tasks like classification and clustering.
- **Anomaly Detection:** Identifying outliers by measuring reconstruction errors on new data.
- **Signal and Audio Denoising:** Cleaning noisy sensor or audio signals, such as in speech or biomedical recordings.

# Advantages

- Help models to learn robust, meaningful features that are less sensitive to noise or missing data.
- Reduce the risk of merely copying input data (identity mapping), especially when compared to basic autoencoders.
- Improve performance on tasks such as image denoising, data imputation and anomaly detection by reconstructing clean signals from corrupted inputs.
- Enhance the generalizability of learned representations, making models more useful for downstream tasks.

# Limitations

- May require careful tuning of the type and level of noise added to the inputs for optimal performance.
- Can be less effective if the noise model used during training does not match the type of corruption seen in real-world data.
- High computational cost, especially with large datasets or deep architectures.
- Like other unsupervised methods, autoencoders do not guarantee that the learned features will be useful for later supervised tasks such as classification or prediction.